

Reinforcement Learning for Player-Specific Strategy Adaptation in Fighting Games



Author: Sandro Megrelishvili

Matriculation: 39851214

Degree: Bachelors of Software Engineering

First Supervisor: Prof. Dr. Iftikhar Ahmed

Second Supervisor: Shan Faiz

Submission Date: July 10, 2025

**Department of Tech and Software Engineering
University of Europe for Applied Sciences,
Potsdam**

EIGENSTÄNDIGKEITSERKLÄRUNG / STATEMENT OF AUTHORSHIP

<u>Megrelishvili</u>	<u>Sandro</u>
Name Family Name	Vorname First Name
<u>39851214</u>	<u>Reinforcement Learning for Player-Specific Strategy Adaptation in Fighting Games</u>
Matrikelnummer Student ID Number	Titel der Examsarbeit Title of Thesis/Project

Ich versichere durch meine Unterschrift, dass ich die hier vorgelegte Arbeit selbstständig verfasst habe. Ich habe mich dazu keiner anderen als der im Anhang verzeichneten Quellen und Hilfsmittel, insbesondere keiner nicht genannten Onlinequellen, bedient. Alles aus den benutzten Quellen wörtlich oder sinngemäß übernommen Teile (gleich ob Textstellen, bildliche Darstellungen usw.) sind als solche einzeln kenntlich gemacht.

Die vorliegende Arbeit ist bislang keiner anderen Prüfungsbehörde vorgelegt worden. Sie war weder in gleicher noch in ähnlicher Weise Bestandteil einer Prüfungsleistung im bisherigen Studienverlauf und ist auch noch nicht publiziert.

Die als Druckschrift eingereichte Fassung der Arbeit ist in allen Teilen identisch mit der zeitgleich auf einem elektronischen Speichermedium eingereichten Fassung.

With my signature, I confirm to be the sole author of the thesis presented. Where the work of others has been consulted, this is duly acknowledged in the thesis' bibliography. All verbatim or referential use of the sources named in the bibliography has been specifically indicated in the text.

The thesis at hand has not been presented to another examination board. It has not been part of an assignment over my course of studies and has not been published. The paper version of this thesis is identical to the digital version handed in.

20.06.2025/ Berlin, Germany

Datum, Ort | Date, Place



Unterschrift | Signature

Abstract

This thesis explores the use of Reinforcement Learning for developing player-specific strategy adaptation in fighting games. The main goal of this thesis is to demonstrate how useful adaptive AI models can be in fighting game environments instead of using hard-coded models, as most of the games do. I used the Gym Retro to emulate the Street Fighter II environment to apply one of the RL algorithms known as Proximal Policy Optimization (PPO) and train an AI agent to learn and adapt to player behaviors. There are two implementations discussed later in the thesis, one of which is trained on 3 million steps, and the other on 7 million. This will demonstrate how crucial the training time can be for an agent. I used Python 3.7.9 to write an actual implementation of the agent, since that version was the only one to support the tools I used, such as Gym Retro, Optuna for hyperparameter tuning, Stable-Baseline3, and PyTorch. The model was trained against scripted AI agents that the game offers. This is one of the many limitations why my implementation is not actually player-specific, but even experimental results demonstrate that the model is capable of improving its ability to perform through continual learning. Other limitations include my hardware setup, which is not ideal for AI training, but is enough for implementing agents to demonstrate basic examples. Despite hardware limitations and the absence of real human opponent data, the trained agent exhibited measurable improvements in combat effectiveness, including basic combo execution, reactive behaviors, and increased win rates. Also, online learning is very challenging to set up and maintain, which is not currently ideal for implementing in video games, especially in fighting games with fast-paced environments. Although the current implementation isn't yet designed to be player-specific, it still provides insights into adaptive gameplay strategies and supports future research directions in AI-driven game designs. This discovery suggests RL's effectiveness in fighting game AI. While PPO is the main focus of this work, the thesis also presents an overview of other prominent RL algorithms to provide a broader understanding and comparative analysis of different approaches.

Table of Contents

Abstract	ii
List of Tables	vi
List of Figures	vii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Research Objective	2
1.4 Research Questions	2
1.5 Contributions	3
1.6 Thesis Outline	3
2 Preliminaries	4
2.1 Reinforcement Learning Basics	4
2.1.1 Agent and Environment	4
2.1.2 Markov Decision Processes	4
2.1.3 Rewards and Policies	5
2.1.4 Exploration vs Exploitation	6
2.2 Common RL Algorithms	6
2.2.1 Deep Q-Network (DQN)	6
2.2.2 Advantage Actor-Critic (A2C)	7
2.2.3 Soft Actor-Critic (SAC)	8
2.2.4 Proximal Policy Optimization (PPO)	8
2.3 Challenges of Applying RL to Fighting Games	9
2.3.1 Real-Time Decision Making	9
2.3.2 Complex Action Spaces	9
2.3.3 Opponent Adaptation	9
2.3.4 Game Engine and Environment Setup	10
3 Literature Review	11
3.1 Introduction	11

3.2	Reinforcement Learning in Games	11
3.2.1	Classical Approaches to Game AI	11
3.2.2	Rise of Deep Reinforcement Learning in Games	12
3.3	Large-Scale RL Projects	13
3.3.1	OpenAI Five (Dota 2)	13
3.3.2	AlphaStar (StarCraft II)	14
3.4	Research Platforms for Game AI	15
3.4.1	FightingICE	15
3.4.2	Other Platforms (Briefly)	17
3.5	Fighting Game AI Research	18
3.5.1	Scripted and Rule-Based AIs	18
3.5.2	RL in Fighting Games	19
3.5.3	Gaps in Existing Work	21
4	Methodology	23
4.1	Overview	23
4.2	Environment Setup	25
4.2.1	Frame Skipping and Reward Shaping	26
4.3	Reinforcement Learning Algorithm	27
4.3.1	Proximal Policy Optimization (PPO)	27
4.3.2	Model Architecture	27
4.4	Training Procedure	29
4.4.1	Data Collection (Rollouts)	30
4.4.2	Policy and Value Update	30
4.4.3	Hyperparameter Configuration	30
4.4.4	Stability Considerations	31
4.4.5	Hardware and Runtime	31
4.5	Evaluation Strategy	32
4.5.1	Baseline Comparison	33
4.6	Player-Specific Adaptation	33
4.6.1	Motivation	33
4.6.2	Limitations	34
4.6.3	Significance and Future Work	35
5	Results and Discussions	36
5.1	Overview	36
5.2	Training Performance	37
5.2.1	Episodic Reward Trend	37
5.2.2	Policy Loss and Value Loss	39

5.3	Quantitative Evaluation	41
5.3.1	Win Rate Against the Baseline	41
5.3.2	Action Diversity	42
5.4	Qualitative Observations	42
5.4.1	Combos and Reactive Behavior	42
5.4.2	Limitations in Complex Scenarios	43
5.5	Opponent Behavior Sensitivity	43
5.5.1	Toward Player-Specific Adaptation	44
5.6	Discussion	45
5.6.1	Strengths of PPO in Fighting Games	45
5.6.2	Challenges Faced	45
5.6.3	Limitations of This Implementation	45
5.7	Summary	46
6	Conclusions	47
6.1	Summary of Work	47
6.2	Key Findings	48
6.3	Limitations	49
6.4	Future Work	50
6.5	Final Remarks	50
	Bibliography	52

List of Tables

3.1 Comparison of platforms for training adaptive fighting game AI . . . 17

4.1 Key Hyperparameters Used During PPO Training 31

List of Figures

4.1	Visualization of the interaction between game environment and PPO agent	24
4.2	Retro Gym Preprocessing Pipepile	24
5.1	Training performance metric for PPO agent at 3 million timesteps. .	37
5.2	Training performance metric for PPO agent at 7 million timesteps. .	38
5.3	3 million steps: Policy Loss	39
5.4	3 million steps: Value Loss	40
5.5	7 million steps: Policy Loss	40
5.6	7 million steps: Value Loss	41

Chapter 1 Introduction

1.1 Background

Artificial Intelligence advances day by day, as if there are no limits to its capabilities. Most people use it for daily tasks to improve productivity, such as using virtual assistants, automated schedulers, or AI-enhanced writing tools. Others use it for entertainment, for example, personalized content recommendations or AI opponents in video games. Although AI gained popularity more recently with models like ChatGPT, Gemini, and even earlier with Apples Siri, the technology has been around for decades. When we look back, we can see examples such as chess engines like Deep Blue [Chess.com \(1997\)](#) or early dynamic NPCs that serve as the groundwork for modern game AI. Even the thesis's main topic, Reinforcement Learning (RL), was theorized over a century ago [Datapod \(2023\)](#).

Reinforcement Learning has become one of the most promising methods for agents to train in the gaming world. Many researchers and developers are experimenting with RL-based agents in games, using different algorithms to measure performance, adaptiveness, and realism. What makes RL particularly useful is that it allows an agent to learn through trial and error. An agent receives rewards for beneficial actions and penalties for mistakes. By implementing this technique, there is no more a need for hard-coded logic or decision trees, since RL offers more autonomy and flexibility to make an agent adaptive and realistic.

As the age of AI is coming to begin, more adaptive models should be used in day-to-day lives to provide a more personalized service. Let's put aside that the idea of this thesis is to have adaptive AIs in fighting games; video games are still a huge part of a lot of people's daily lives, and even if the current implementations are not as good as the theory of player-specific adaptation sounds, developing such models and launching publicly will be one of the biggest steps of AI industry in video games, since the players can actually provide live data, it can help to improve models drastically. Plus, lots of games like criticizing and giving honest feedback, which can also help to optimize models.

1.2 Problem Statement

Even though we already have technology to mimic human behavior in video games, most game AIs today are static and rely on predefined difficulty levels or hard-coded behavior. They do not adapt to the unique style of a specific player, which creates a gap in realism and engagement.

Let's take an example of one of the most popular games I used to play often, Tekken 7. I went through story mode and played against hard-coded models lots of times, but unfortunately, it does not feel very challenging or engaging after some time. Even though it is mainly developed to be a multiplayer game to play against other players, it would still be very helpful to have an adaptive AI model in-game. It can help a lot of gamers to evolve as players and try to overcome themselves. To compare it to other genres, for example racing game, AI isn't necessary to practice on laps, but the good part of those is that in practice laps, there sometimes is a ghost of your past self, which performed the best. That ghost pushes you to be better at that part of the game and get closer to your limits. Therefore, fighting games should have the same idea.

1.3 Research Objective

With this thesis, I want to close the gap by allowing AI agents to adapt their play style to specific players. As I already mentioned, my current implementation is far from what my thesis suggests, but it still lays the groundwork for future advancements toward customized gameplay in video games, especially fighting games.

My version of the ideal fighting game environment would be to have an opponent which know how you play and helps you improve your playstyle. Based on the observations it will do, it can practically predict your next move and will counter you more as the real-life pro players do. Theoretically, this type of model can become unbeatable, but that's the reason why it can be an improved environment to train your skills.

1.4 Research Questions

To guide the direction of this thesis, several research questions were formulated. These questions aim to explore the feasibility, effectiveness, and challenges of using reinforcement learning for adaptive behavior in fighting game environments.

- Can Reinforcement Learning be used to train a fighting game AI that adapts to different opponent behaviors?

- Which type of RL algorithm is more effective for this task?
- What are the key challenges in developing player-specific strategy adaptation in fighting games?

1.5 Contributions

Since I implemented an AI agent for the iconic fighting game Street Fighter II using RL, it is the main subject of this thesis. Among various RL algorithms, I chose Proximal Policy Optimization (PPO) due to its balance between stability and performance in complex environments. By avoiding significant policy changes that could disrupt learning, PPO allows agents to progressively improve their tactics. The model gradually learns the best behaviors, like blocking attacks, punishing whiffs, and executing combos according to the opponent's style, through direct interaction with the game environment.

Training an AI agent for fighting games is especially challenging. These games require rapid decision-making, involve complex move sets, and require constant strategic adaptation. Despite these difficulties, fighting games are ideal testing environments for studying how artificial agents can evolve to simulate human-like reflexes and adaptability. Fighting games require quick decision-making and strategic depth, unlike turn-based games such as chess, where decisions are carefully planned. However, this creates an ideal environment for AI to simulate human reflexes and adaptability.

1.6 Thesis Outline

I will also briefly go through and compare other well-known RL algorithms like Deep Q-Networks (DQN), Advantage Actor-Critic (A2C), and Soft Actor-Critic (SAC) in order to give a deeper understanding of its potential and limitations in fighting game AI. The comparison of different RL algorithms will help you to understand PPO's strengths and realize why it can be the best-performing one in the fighting game environment.

Lastly, I will suggest how this project can be improved in the future to include mechanisms for capturing people's playstyles and allow AI to evolve specifically to different opponents. This thesis supports the long-term goal of developing a fully engaging AI-powered gaming experience.

Chapter 2 Preliminaries

2.1 Reinforcement Learning Basics

2.1.1 Agent and Environment

In Reinforcement Learning, the most important assets are the agent and the environment (Sutton & Barto 2018, p. 47-48). The agent is the decision maker, while the environment is everything this agent interacts with. For example, in my implementation, the Agent was the player, and the environment was the box fights took place, and that box also included the opponent, which is also part of the environment.

At first, the actions of the agents are random and based on what it does, they receive either a reward or a penalty. If the agent does something beneficial for itself, for example, hitting its enemy or winning a fight, it receives a reward. But if it loses its health or even a fight, then a penalty is applied in the form of negative points. In my Street Fighter II implementation, the agent gets around 100 to 500 points based on how much damage it dealt to an enemy, and after winning a match, it receives multiple 1000 points to show that winning is the most rewarding.

After multiple attempts of its randomized movements, it gets an idea of which type of playstyle is beneficial for its matches; Therefore, in the next fight, it acts accordingly. This strategy is also called a policy that makes sure the agent is making decisions to maximize its long-term rewards. For the agent to actually know how to play, it needs to train thousands or even millions of times.

2.1.2 Markov Decision Processes

If you want to demonstrate an agent's training process mathematically, then Markov Decision Process (MDP) is a great way to do so. This mathematical framework calculates how decisions are made based on its current state and a bit of randomness.

In reinforcement learning, the agent and environment interact at discrete time steps $t = 0, 1, 2, \dots$. At each time step t , the agent receives a state $S_t \in \mathcal{S}$, selects an action $A_t \in \mathcal{A}(S_t)$, receives a reward $R_{t+1} \in R$, and transitions to a new state

S_{t+1} . This process gives rise to a trajectory:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

(Sutton & Barto 2018, p. 47-48)

A Markov Decision Process (MDP) is defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where:

- $\mathcal{S}$ is a finite set of environment states.
- $\mathcal{A}$ is a finite set of actions available to the agent.
- $\mathcal{P}(s', r \mid s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}$ is the transition dynamics function.
- $\mathcal{R}(s, a)$ is the expected reward after taking action a in state s .
- $\gamma \in [0, 1]$ is the discount factor, which prioritizes immediate rewards over distant ones.

The Markov property assumes that the next state and reward depend only on the current state and action, not on the full history:

$$\Pr(S_{t+1}, R_{t+1} \mid S_t, A_t) = \Pr(S_{t+1}, R_{t+1} \mid S_0, A_0, R_1, \dots, S_t, A_t)$$

In short, the Markov property says that what an agent will do next is dependent on the current state and action, instead of the whole past history. This allows RL algorithms to optimize their policies efficiently in environments where the Markov property holds.

2.1.3 Rewards and Policies

As we already mentioned, there are two key factors while training the agent: **rewards** and **policies**.

The rewards can be either positive or negative. Positive rewards are gained when the agent does something beneficial for itself, such as damaging its opponent or winning a fight. Depending on the type of action the agent takes, the numerical value of the reward will be calculated accordingly. Negative rewards, also called *penalties*, are acquired when the agent performs actions that are unproductive or unnecessary.

The second factor, the **policy**, is the agent's behavior pattern. In Reinforcement Learning, the policy guides the agent on what action to take in a given situation. During training, the agent is essentially learning and refining this policy. Initially, its behavior is random, but after multiple attempts, the model improves its policy

based on the feedback received through rewards. Over time, it develops a better understanding of which actions lead to higher rewards and fewer penalties.

In short, the ultimate goal of the agent is to learn the best possible policyone that results in consistently high rewards over time. This is achieved by repeatedly interacting with the environment, experimenting with different actions, and learning from the outcomes. After many trials, the agent identifies the most effective strategies for reaching its goal.

2.1.4 Exploration vs Exploitation

One of the biggest challenges in Reinforcement Learning is to find a balance between **Exploration** and **Exploitation**. This is also known as the Exploration vs Exploitation dilemma. [Yerremshetty \(2025\)](#)

Exploration is when the agent tries different things it has not done before. Basically, the name already tells us that it is exploring different move sets, combos, and actions. On the other hand, Exploitation is when the agent tries actions it already knows. For example, my model noticed that throwing a low kick was beneficial, so it started using that in every fight on purpose.

Both are important, but there should be a limit on each of them. If the agent decides to always explore, then it will not do as good of a job as it should have. Also, if it did not explore enough, then this movement will be very limited since it might not have tried out better methods yet. So, there should be a balance. The agent should explore more at the beginning and then slowly start to exploit what it has learned.

2.2 Common RL Algorithms

Reinforcement learning includes many different algorithms, each with its own use cases. Some are suited for simple games with clear outcomes, while others perform well in complex environments like fighting games. This section includes four major algorithms that are used by RL agents.

2.2.1 Deep Q-Network (DQN)

One of the most well-known reinforcement learning algorithms is Deep Q-Network (DQN), which was developed by DeepMind to combine Q-learning with deep neural networks. Traditional Q-learning stores values in a Q-table, which is good for small states, but for complex environments, for example, video games, states are too large to be stored in a table. Therefore, DQN offers a neural network. This allows an agent

to handle high-dimensional inputs like game frames and still learn which action to take.

Each time the DQN agent tries to interact with the environment, it first observes the current state. Then, it picks an action, which is usually chosen by a greedy policy to balance exploration vs exploitation. After all that, it receives a reward and observes the next state, and the loop continues. The agent stores its past experiences in a replay buffer, so in the future, it will take samples from this buffer and train the neural network to minimize the error between the predicted Q-value and the target Q-value.

DQN became famous for its success in Atari games. It learned how to play by observing and trying out different moves. For example, in the game Breakout, the paddle moves left or right to bounce a ball and destroy bricks. The state is usually represented as a few stacked greyscale frames, and the actions are limited. This worked very well here since the game did not include a complex environment, its move set was relatively small, and the environment was fully observable. But that is not the case in fighting games. When it comes to demonstrating high-dimensional actions, which involve complex combos, DQN is not really effective. [Luu \(2025\)](#)

2.2.2 Advantage Actor-Critic (A2C)

Advantage Actor-Critic (A2C) works a slightly different way. It actually combines two different approaches: Policy-based methods, which directly learn a policy, and Value-based methods, like the one in DQN, which estimate how good each state or action is. But what really makes A2C different from DQN is its method to use two models: Actor and Critic.

The Actor is the policy network that decides which action to take. On the other hand, the Critic is the value network that evaluates how good the current state or action is. While the actor explores strategies, the critic provides feedback on whether the chosen actions are helpful or not.

What makes A2C the improved version of the standard Actor-Critic approach is the advantage function, which helps the agent focus on actions that are better than average in a given state.

In A2C, the agent observes the current state. Then the actor picks an action based on the current policy. After all that, the environment returns a reward and moves on to the next state. In the meantime, the critic estimates the value of the current and next states. The advantage is calculated, and both networks, actor and critic, are updated using gradient descent to minimize their respective losses. [PyLessons \(2020\)](#)

2.2.3 Soft Actor-Critic (SAC)

Soft Actor-Critic (SAC) is another actor-critic-based algorithm, but in this case, the agent also learns from past and stored experiences. It is especially useful in continuous action spaces and is known for achieving both high sample efficiency and stable training.

What makes SAC special is the use of entropy in its objective function. This algorithm doesn't just aim to maximize its reward like other algorithms do. It also tries to maximize the entropy of the policy, which means that the agent is encouraged to explore more by choosing more random and less deterministic actions at the beginning of training. This type of approach also helps the agent to avoid getting stuck with a poor strategy too soon. For example, when I was training my model with another approach, my agent got stuck on a very bad policy. Instead of focusing on attacking and gaining rewards, it just tried to back off and jump to dodge attacks.

As I already mentioned, SAC is very useful in continuous action spaces, and because of that, it does not naturally fit with the video game environment. It is mostly designed for real-life robotics and mechanisms with a more complex environment, but with correct modifications, it can also be used in fighting games. [Haarnoja et al. \(2018\)](#)

2.2.4 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is one of the most popular RL algorithms today because it is designed to be simpler to implement, more stable, and more sample-efficient [Schulman et al. \(2017\)](#). And as I already mentioned, this is the method I chose to go with when I trained my Street Fighter II agent.

This RL algorithm also belongs to the family of policy gradient methods. It uses the concept of clipping in its objective function to control how much the policy can change during an update. At first, the agent interacts with the environment and collects experience, such as state, action, reward, and next state. Then, it calculates how good the action was using a value function called the advantage. Finally, using the clipping ratio of new to old probabilities, PPO updates the policy only if the new action probabilities are not too far from the old ones. Basically, PPO tries to improve the policy but prevents over-adjusting in a single step, which leads to more stable training.

In this thesis, PPO is the main algorithm because of its strong performance in high-speed, high-dimensional settings. Also, due to its learning approach, PPO is well-suited for challenging environments with lots of possible states and actions,

which is exactly what happens in fighting games.

2.3 Challenges of Applying RL to Fighting Games

Fighting games, like Street Fighter II, have a very unique set of challenges for reinforcement learning. Unlike simpler environments, fighting games are fast-paced, highly reactive, and require precise timing. They have a large and complex action space, predictable opponent behavior, and constant real-time decision-making. All of these factors make it more difficult for an RL agent to learn effectively and perform well. This section will explain the main challenges that come up when using reinforcement learning in fighting games.

2.3.1 Real-Time Decision Making

As I already mentioned, fighting games are fast-paced and need very accurate reactions. This differs a lot from other RL environments, such as chess, where the agent has time to think. But in fighting games, the agent needs to make decisions every few frames. Not only should it learn optimal moves and counterattacks, but it should also be able to time it perfectly, which is not an easy task even for a trained human. The slightest mistake can lead to heavy penalties or missed opportunities, especially when executing complex combos, parrying, or blocking attacks.

2.3.2 Complex Action Spaces

I already described the fighting game environments as complex and also mentioned a few algorithms that can handle such complexity. Even though managing the fighting game environments is possible, it does not mean that it is not a challenging thing to do. Even if we put aside its constant movement, there are still a lot of potential inputs involved, such as jumping, crouching, blocking, and multiple ways of attacking. You can even build combos that sometimes involve a few actions to perform simultaneously. After all that, the agent should also take into account its position, the enemy's position, and other factors depending on the game. Therefore, just the game's complex action space is a very challenging task for the agent to master.

2.3.3 Opponent Adaptation

Fighting games involve constant strategy adjustments based on the opponent's behavior. The opponent in fighting games is often another intelligent agent, either a human or an AI, that may change strategies mid-match. Usually, trained models are

designed for a specific strategy. If they can perform really well, but once the strategy is changed, their performance falls. Therefore, in my opinion, it is important for the agent to be adaptable to make the game more interesting, less predictable, and fun to play against.

2.3.4 Game Engine and Environment Setup

Training a reinforcement learning agent requires access to the underlying game state and the ability to simulate interactions efficiently. In this thesis, the *Street Fighter II* game environment is accessed via the Retro Gym framework, which emulates the game and exposes its internal states and control inputs to the learning agent.

Chapter 3 Literature Review

3.1 Introduction

Reinforcement learning has emerged as one of the most powerful tools in the development of AI, including game-playing AI. From simple rule-based systems in early arcade games to highly advanced agents capable of surpassing humans. This chapter presents an in-depth review of existing research and technologies used in various games, particularly those with complex environments.

The goal of this literature review is to provide a clear understanding of how RL has been applied in the gaming industry. First, we will explore the traditional methods like finite-state machines and behavior trees, and then move to more complex, learning-based systems. This chapter will not only review the fighting game research, but also other types of genres to have a better understanding of how RL can be applied to different environments.

3.2 Reinforcement Learning in Games

3.2.1 Classical Approaches to Game AI

Before Machine Learning and Reinforcement Learning were popular, most game AIs had hard-coded logic to control non-player characters (NPCs). Since game environments have been relatively simple, designing such behaviors was the best solution, but such techniques were limited in adaptability and scalability.

One of the most popular approaches was the finite-state machine (FMS). In an FMS, the agent switched between a predefined set of states based on conditions triggered by in-game events. Designing and debugging FMS is very simple, which makes it ideal for early arcade and console games. However, since games got more complex, managing agents' behavior got difficult too, particularly in games that require nuanced strategy or long-term planning.

Another popular method is the behavior tree, which is a hierarchical model used to control the behavior of characters in games. Behavior trees allow for more

modular and reusable code compared to FMSs, which allow developers to design more sophisticated and flexible behaviors. Even though behavior trees don't support learning from experience, they are still widely used in modern games like Halo 2 [Isla \(2005\)](#).

Decision trees and utility-based systems have also been implemented in classical game AI. A decision tree is also a hierarchical model that maps specific input features, like health or enemy distance, to predefined actions. On the other hand, the utility-based systems assign scores to possible actions based on current game conditions and choose the action with the highest utility. Both of these approaches offer more flexibility and can simulate prioritization in decision-making, but they still do not support adaptability to new scenarios.

Many games have also relied on scripted AI. While implementing this approach, developers should explicitly define how NPCs behave under different circumstances. This technique is nice to have in story games, but the agent is still highly predictable and less challenging.

3.2.2 Rise of Deep Reinforcement Learning in Games

The rise of deep reinforcement learning (DRL) has dramatically transformed how artificial intelligence is used in video games. By combining traditional reinforcement learning with deep neural networks, DRL allows agents to learn and operate in complex environments that provide high-dimensional and unstructured data, such as raw pixels from game screens. This marked an important shift from classical AI techniques towards agents that can improve their behavior just by engaging with the game environment.

The creation of DeepQ-Network was one of the most important achievements in the history of DRL. In 2015, this RL algorithm of DeepMind successfully learned how to play Atari 2600 games directly from raw pixel inputs [Mnih et al. \(2013\)](#). DQN demonstrated that a deep neural network is able to approximate the Q-function, which allows agents to learn effective policies without any hard-coded behavior. A target network and experience replay were used to help stabilize training and operate as fundamental design decisions for many DRL techniques today.

While DQN was effective in unique action spaces, it still struggled with continuous control problems. To solve this, researchers developed actor-critic algorithms, such as Deep Deterministic Policy Gradient (DDPG). Since we already explained how actor-critic algorithms work, we now know that it is a great method to extend DRL's applicability to environments with continuous actions.

Other algorithms, such as Asynchronous Advantage Actor-Critic (A3C) and Proximal Policy Optimization (PPO), improved performance and stability in com-

plex environments. These methods introduced better policy updates and parallel training strategies, which made DRL stable and scalable.

Over time, DRL has been successfully applied far beyond arcade games. It now powers agents in board games like Go and in large-scale real-time strategy games like StarCraft II. These applications have shown that DRL agents can deal with long-term planning, partial observability, and vast action spaces challenges that were once considered major obstacles for AI.

Overall, the development of deep reinforcement learning has made it possible to create game agents that are more intelligent, flexible, and human-like. It has become the most popular approach for research in complex environments and serves as the foundation for the methodology explored in this thesis.

3.3 Large-Scale RL Projects

3.3.1 OpenAI Five (Dota 2)

Since PPO is the main approach for this thesis, I wanted to mention one of the most significant achievements in this field, which is Dota 2 AI. Even though it is not a fighting game, the environment of a multiplayer online battle arena (MOBA) is still very complex. The project OpenAI Five, developed by OpenAI, demonstrated how well a deep reinforcement learning agent handles highly complex, dynamic, and team-based environments.

Dota 2 poses a unique challenge for AI. It's a real-time strategy game that requires quick decision-making, long-term planning, coordination with teammates, resource management, and the ability to adapt strategies mid-game. In this complex environment, matches can last up to 40 minutes with plenty of different moves at any moment, and not only do traditional AI models but even humans struggle to cope. To handle this, OpenAI used Proximal Policy Optimization (PPO), a reinforcement learning algorithm known for its stability and scalability. In order to continuously improve, the agents were trained by self-play, which involves playing against previous versions of themselves. By using this technique, the agent was able to create counterplays and learn complex strategies without requiring humans to play against it.

What's truly remarkable is the training infrastructure, where Dota 2 was running. OpenAI used thousands of parallel game simulations, running on hundreds of GPUs and CPUs. That is worth over 180 years of experience in a day [Berner et al. \(2019\)](#). This powerful machine was able to teach the agent lane control, item timing, and coordination during team fights typically associated with experienced players.

The demonstration matches of this project were truly magnificent. OpenAI Five played against top players, including former world champions, and in April 2019, it achieved a historic victory by defeating a team of world-class Dota 2 players. Even though the game had some restrictions, such as limited hero choice, it still highlights the potential of deep reinforcement learning in mastering complex and competitive games.

OpenAI Five also demonstrated emergency behaviors, such as baiting enemies, initiating fights, and retreating to heal. All of this without any hard-coded actions. These behaviors proved that large-scale self-play models can imitate human-like strategies only through reinforcement learning.

The OpenAI Five project stands as a significant advancement in reinforcement learning research, showing how a well-designed algorithm and extensive training infrastructure can create highly competent and generalizable agents in one of the most challenging game environments to date.

3.3.2 AlphaStar (StarCraft II)

Another great achievement in large-scale reinforcement learning is AlphaStar, developed by DeepMind. This project was created to play StarCraft II, one of the most strategically complex games in existence. Unlike turn-based games or environments with limited action spaces, StarCraft II has long time horizons, thousands of actions per minute, partial observability, and high-level strategic planning over extended gameplay. Because of these characteristics, it is a perfect but extremely challenging environment for artificial intelligence research.

AlphaStar began its training using supervised learning, where it imitated human players by learning from a large dataset of professional replays. This allowed the model to gain a fundamental knowledge of gameplay mechanics, macros, and micro decision-making, and standard build orders. However, to surpass humans, it still required further improvements using deep reinforcement learning [Vinyals et al. \(2019\)](#).

The second phase of training is almost like it was in OpenAI Five. It involves multi-agent reinforcement learning, where a large number of agents play against each other and co-evolve. This approach introduced diversity in strategies and prevented sticking to any single tactic. It also encouraged the development of some counter-strategies. Each agent adapted to different opponents, which led to a form of curriculum learning driven by internal competition.

DeepMind used several advanced strategies to manage the massive complexity of StarCraft II:

- **Value-based and policy gradient methods** combined with neural net-

works to estimate game value and action probabilities.

- **Transformer-based architectures** to handle the spatial and temporal components of the game.
- **Restricted action spaces** that gradually expanded, allowing AlphaStar to scale from micro-level control to macro-level planning.

In January 2019, AlphaStar defeated top-ranked professional players in matches that were publicly streamed. It achieved Grandmaster level on the official StarCraft II ranking, placing it within the top 0.15 percent of human players. In addition, the final system functioned under human-like limitations, like limited camera access and realistic reaction times, which improved the results’ realism.

AlphaStar’s development demonstrated the potential of combining imitation learning, population-based training, and reinforcement learning to solve complex, long-term problems in multi-agent environments. Furthermore, it demonstrated how well-structured training paradigms and scalable learning architectures can give rise to emergent coordination and strategy.

The success of AlphaStar further highlighted the feasibility of applying RL in real-time, partially observable, high-dimensional environments, setting the stage for research in domains such as robotics, autonomous systems, and competitive games like fighting games.

3.4 Research Platforms for Game AI

3.4.1 FightingICE

FightingICE, in my opinion, is the best example of AI-driven fighting games. FightingICE is a fighting game AI competition engine. It is a research-focused platform specifically designed for developing and evaluating artificial intelligence in 2D fighting games. Created at Ritsumeikan University in Japan, FightingICE has become a widely adopted testing ground in the AI research community, particularly through its use in the fighting game AI competition [Matsumoto et al. \(2018\)](#).

The platform simulates a fighting game environment where two agents are fighting in real-time using predefined moves, such as punches, kicks, jumps, and blocks. It provides a controlled but also challenging domain for testing AI behavior. It offers several important features that make it suitable for reinforcement learning and game AI experiments:

- **Standardized Interface:** FightingICE enables researchers to apply and assess AI without having to change the game engine itself, by offering a stan-

dardized API for creating agents. This interface is adaptable to a variety of approaches since it supports both learning-based and scripted agents.

- **Deterministic Environment:** FightingICE functions in a deterministic environment, compared to many commercial fighting games that add randomness or frame delay. In reinforcement learning, where consistency of state transitions is essential, this enables reproducibility and stable training.
- **Real-Time Decision Making:** FightingICE enforces real-time constraints, which require agents to choose actions within time budgets. This puts pressure on AI systems to process states efficiently and make decisions quickly, which aligns with real-world conditions.
- **Built-in Evaluation Metrics:** FightingICE includes performance-tracking tools like win and loss statistics, total damage dealt, and remaining health. These metrics support tracking learning progress over time and measuring the effectiveness of AI strategies.
- **Low Latency:** FightingICE’s lightweight architecture allows simulation episodes to run fast. This makes it ideal for reinforcement learning, where millions of simulation steps are often run during training.
- **Competitions and Benchmarks:** FightingICE is used as the official engine for fighting game AI competition, so the platform is attracting participants from around the world. These competitions offer standard tasks and benchmark agents, allowing for a comparative analysis of different AI approaches and algorithms.

FightingICE has been used in a variety of academic studies, including those focused on behavioral modeling, opponent adaptation, hierarchical reinforcement learning, and motion planning [Togelius et al. \(2011\)](#), [Ikeda & Arita \(2018\)](#). Its structure encourages experimentation with both low-level motor control and high-level decision-making, making it ideal for exploring challenges unique to fighting games, such as combo execution, state prediction, and adaptive gameplay.

In this thesis, FightingICE is used as a reference, inspiration, and contrast to other environments like Gym Retro. Commercial game emulators, such as Street Fighter II via Gym Retro, add complexity and realism, better simulating real-world gameplay conditions, while FightingICE offers a clear and deterministic environment perfect for algorithmic comparisons.

As previously discussed, FightingICE offers a deterministic environment and standardized API. This makes it ideal for studying opponent adaptation, a core

requirement for player-specific RL agents. However, its lack of human player data limits its utility for training personalized AI.

Table 3.1: Comparison of platforms for training adaptive fighting game AI

Platform	Strengths	Weaknesses
FightingICE	Deterministic, benchmarked for competitions	Limited access to human gameplay data
Gym Retro	Real-world game complexity, easy setup	No built-in support for opponent modeling
Unity ML-Agents	Highly customizable, supports human input	Requires significant setup for fighting game mechanics

Tale 3.1 shows strengths and weaknesses of popular training platforms such as FightingICE, Gym Retro(implemented in this study), and Unity ML-Agents. To be honest, Unity ML-Agents is probably the best option from here because of its highly customizable environment, but it is also the hardest to set up. Because of the limited time for research, implementations were done in Gym Retro, which is the easiest to begin with; therefore, Gym Retro is the most beginner-friendly from the list. FightingICE, on the other hand, is the benchmark of all fighting game environments. It stands in the middle of other training platforms, which makes it ideal for research purposes. The research was still done in Gym Retro since one of the research purposes is for beginners to understand the methodology easily.

But FightingICE’s success with a deterministic environment motivated our use of frame skipping and reward normalization in Gym Retro to stabilize training. Future interactions of FightingICE could integrate human gameplay datasets or APIs for real-time opponent modeling, bridging the gap between controlled benchmarks and adaptive in-game AI.

3.4.2 Other Platforms (Briefly)

Despite FightingICE being a leading platform for research in fighting game AI, there are multiple other environments that contribute a significant amount to reinforcement learning research across a large number of game genres. Such platforms provide settings of increased complexity and diversity, resulting in better evaluation of generalization, adaptability, and scalability of learning algorithms.

OpenAI Gym Retro - Gym Retro, developed by OpenAI, is a framework that allows reinforcement learning agents to interact with thousands of classic video games, among which are games from platforms such as Sega Genesis, Nintendo Entertainment System (NES), and Super Nintendo (SNES). It is built on top of the Retro emulator and is fully integrated with the OpenAI Gym API [Nichol et al. \(2018\)](#). A particularly notable use of Gym Retro is training agents to play Street

Fighter II, offering a real commercial fighting game equipped with visual complexity, limited observability, and large discrete action spaces. There are some challenges, however, which include: Sparse rewards depending on game progression, delayed outcomes for strategic moves, noisy and partially observable pixel-based input. In spite of these difficulties, Gym Retro enables experimentation with reinforcement learning in gaming scenarios, which are considerably more authentic and visually diverse, including fighting games that are closer to real player experiences than controlled simulators like FightingICE.

Project Malmo (Minecraft) - Project Malmo is a research platform developed by Microsoft that enables reinforcement learning agents to interact with Minecraft, a 3D sandbox game with open-ended tasks and goals. The environment supports visual and symbolic state representations alike, being designed for evaluating long-term planning, navigation, and agent collaboration in complex, multi-agent worlds. Malmo is not focused on combat mechanics like fighting games, however it is very relevant for its support of adaptive behavior in large, partially observable spaces.

Unity ML-Agents - The Unity ML-Agents Toolkit is another general-purpose platform that enables researchers to train agents in environments developed with the Unity game engine. This platform also supports a vast array of game genres and contains customizable fighting environments, continuous control tasks, and platformers. Unity ML-Agents is particularly useful for: creating visually realistic simulations, rapid prototyping of complex multi-agent scenarios and supporting both discrete and continuous action spaces. It is also not specifically tailored to fighting games, however its flexibility makes it a majorly capable tool for testing general reinforcement learning algorithms and adaptive behavior. These platforms collectively contribute to the field, offering different perspectives on AI behavior, starting from tightly controlled simulations (FightingICE), to commercial-grade environments (Gym Retro), to fully customizable worlds (Unity). Such a diverse conglomeration of tools assists the advancement of the development of more generalizable and robust AI agents capable of performing in real-world settings.

3.5 Fighting Game AI Research

3.5.1 Scripted and Rule-Based AIs

Before learning-based approaches became widely adopted, most fighting game AIs utilized scripted logic and rule-based systems. While these methods were in fact efficient and easy to implement, they could not remotely compare to human players or reinforcement learning agents in terms of adaptability and strategic depth.

In traditional fighting games, AI opponents are often constructed with Finite-

State Machines (FSMs), decision trees, or behavior trees. Each one of the aforementioned structures encodes pre-defined sequences of actions based on specific game states. We can demonstrate this with an example: An enemy AI might be programmed to jump when a projectile is detected and the player is within a certain distance or block of it. These actions are usually governed by deterministic rules written by human designers.

These approaches are reliable and offer predictable difficulty levels; however, they do suffer from multiple crucial limitations:

1. **Lack of Adaptation:** Scripted AIs follow fixed patterns and lack the ability to adjust to the behavior of the player. Once a player recognizes these patterns, the opponent becomes predictable and easily exploitable.
2. **Static Difficulty Tiers:** Many games implement multiple difficulty models (e.g., Easy, Normal, Hard), but these levels usually only alter aggressiveness or reaction times, rather than modifying strategic behavior. This results in the differences between difficulty levels being shallow.
3. **No Learning Mechanism:** Such systems do not learn or evolve over time. They cannot incorporate player-specific data or past performance into future decision-making, limiting their long-term engagement value.

In spite of their drawbacks, rule-based systems have historically maintained their popularity because of their simplicity, controllability, and performance efficiency. Even current commercial fighting games sometimes utilize hybrids, which combine rule-based control for basic behaviors with machine learning elements for more complex decision-making. Scripted AI also takes on the function of baseline agents in AI research competitions, an example being the Fighting Game AI Competition. They have been proven useful for evaluating the effectiveness of reinforcement learning agents, as they supply a stable and predictable benchmark for training and comparison.

To summarize, rule-based systems have shaped the early design of fighting game AIs, but their lack of flexibility and adaptability has created a need for more advanced approaches, such as reinforcement learning. These dynamic techniques offer the potential for emergent behaviors, personalized strategies, and non-repetitive gameplay features, which are critical for guaranteeing player engagement in competitive games.

3.5.2 RL in Fighting Games

Reinforcement learning has demonstrated good results in many kinds of gaming environments, but there are specific advantages and disadvantages to applying RL

to fighting games. As we already mentioned, fighting games are a compelling domain for testing adaptive, learning-based agents because of their high-dimensional spaces, dynamic opponent behavior, and fast-paced, real-time decision-making.

Applications and Algorithms - The application of RL in fighting games has been the subject of many studies, often involving platforms like FightingICE and Gym Retro. Proximal Policy Optimization and Deep Q-Networks have been two of the most popular algorithmic techniques. Agents have been trained using these techniques to learn from pixel or state-based inputs and modify their strategies over time. For example, in the fighting game AI competition, many competitors submit PPO or A2C-trained agents. Despite there still being issues with generalization and robustness, these agents demonstrated effective performance against FightingICE’s built-in rule-based agents.

Opponent Modeling and Adaptive Difficulty - An important part of RL research in fighting games is opponent modeling, where the agent tries to anticipate and react to various players’ strategies. It has been suggested that adaptive opponents can recognize player patterns in real time and modify their strategies accordingly, which is compatible with the goal of player-specific adaptation. Online learning and meta-RL approaches, in which agents continuously adjust to new opponents instead of depending only on offline training, have been studied in some work. By avoiding repetitive or easily exploitable behaviors, a common criticism of traditional scripted AIs, this adaptability increases player engagement. Additionally, in order to manage the complex structure of fighting game actions, where one policy controls low-level movement and another manages high-level decisions, researchers have suggested employing hierarchical RL.

Environment Setup and Reward Shaping - Fighting games often require careful reward shaping to guide learning agents. Unlike turn-based games with discrete win/loss conditions, fighting games require agents to make hundreds of micro-decisions during a match. Damage dealt or health remaining are examples of simple reward functions that are often too limited or delayed to encourage effective learning. In order to prevent this, researchers implement intermediate rewards that are dependent on successful attacks, combos, or defensive actions. This lets agents get more insightful feedback as they train. To create a more realistic and demanding learning environment, some studies also experiment with frame skipping, action delay, and partial observability. These modifications imitate real-world scenarios in which agents are unable to observe game states perfectly or respond to them instantly.

Limitations and Open Challenges - RL in fighting games still has a number of limitations, despite encouraging improvements:

1. **Overfitting to fixed opponents:** Agents trained against specific strategies often struggle to generalize to novel behaviors or human players.
2. **Slow training times:** Training takes a lot of episodes because fighting games are so complex and comprehensive.
3. **Exploration vs. exploitation:** Agents often struggle to balance between trying new strategies and refining existing ones, especially in games with deceptive reward landscapes.

These challenges highlight the need for continued research into adaptive learning, sample-efficient algorithms, and robust generalization methods.

3.5.3 Gaps in Existing Work

Even though the use of reinforcement learning in fighting games has advanced significantly, there are still a number of issues with the current state of it. These gaps highlight areas that require more research in order to create agents that are more intelligent, engaging, and adaptable, especially those that are designed for player-specific, real-time interactions.

Lack of Player-Specific Adaptation - Most RL-based fighting game agents are trained to perform well against a fixed set of opponents, whether they are rule-based, random, or trained agents. While this setup offers a consistent training environment, it ignores the variety of human play styles. General-purpose agents are unable to capture various patterns, preferences, and habits that human players often show. As a result, a lot of trained agents perform well in static matchups but have trouble with new or unpredictable strategies. The difference points out the necessity of player-specific adaptation, in which agents continuously pick up new skills and modify their strategies in response to the actions of particular players.

Limited Use of Real-Time Feedback - Most implementations do not include real-time behavioral feedback from the opponent, despite knowing that RL agents receive rewards during training. Through their in-game actions, human players may unintentionally evoke difficulties, boredom, or frustration, signals that could be used to dynamically modify the agent’s approach. By using these real-time indicators in the learning process, adaptive difficulty systems that adapt more naturally to the player’s changing skill level may become possible.

Overemphasis on Win Rate and Performance Metrics - Many existing RL studies optimize agents solely for performance-based objectives like maximizing win rate or damage dealt. Even though these metrics are important, they fall short in assessing the quality of gameplay from a perspective of the human experience. An agent that frequently wins by spamming a single move, for example, could get a

large reward but offer an average and boring gaming experience. It is becoming more widely acknowledged that training objectives should also take into account gameplay quality, engagement, and balance. Implementing these human-centric factors into RL reward functions is not explicitly explored in many studies.

Generalization and Robustness Issues - Agents must often generalize across different kinds of characters, stages, and strategies in fighting games. Many RL agents, on the other hand, are very specialized and have been trained in environments with little to no diversity. They are therefore not as strong when used in new environments or against invisible opponents. This limits their applicability in dynamic, real-world contexts, particularly in online or multiplayer games. To increase generalization and guarantee broader applicability, meta-learning, domain randomization, or population-based training methods are obviously required.

Underexplored Hierarchical Control and Strategy - At last, even though there has been some recent research on hierarchical reinforcement learning in games, there hasn't been much of an impact on fighting games. Because actions take place at several levels of abstraction, from quick movements to mid-level tactics and high-level strategies, fighting games are naturally hierarchical. Agents capable of thinking at these levels may be able to behave more like skilled humans. Current RL systems frequently function on a flat action space, missing out on opportunities for more calculated and understandable choices.

By addressing these research gaps, future work can contribute to the development of more intelligent, responsive, and human-aligned fighting game agents. In order to improve the customization and unpredictable nature of AI opponents in classic fighting game environments like Street Fighter II, this thesis focuses on player-specific strategy adaptation.

Chapter 4 Methodology

4.1 Overview

This chapter explains the technical methodology used to create a reinforcement learning agent capable of playing Street Fighter II. The primary goal is to utilize the Proximal Policy Optimization (PPO) algorithm to enable the agent to continually interact with the game environment, thereby learning efficient combat strategies. The main goal is to use the Proximal Policy Optimization algorithm to help the agent constantly interact with the game environment in order to learn efficient combat strategies. The objective is to develop an agent that can potentially adapt to specific player behaviors as well as master basic skills.

The methodology is structured around a number of essential elements, including the environment setup, which serves as the interface between the agent and the game, the RL model architecture is designed to process complex game states, training procedures optimize the agent's policy, and the evaluation strategies are used to assess the behavior and performance of the agent.

The theory behind algorithm selection is explained in this chapter, along with changes that were made to the action and observation spaces to balance complexity and learnability. The training pipeline, including reward design and performance monitoring, is also covered. Lastly, an initial strategy for player-specific adaptation is provided, laying the groundwork for further research in AI for personalized fighting games.

In figure 4.1 game emulator represents the Retro Gym environment, which simulates the Street Fighter II game and produces raw frame observations and in-game information. Preprocessing is processing raw RGB frames into grayscale conversion and conversion into the shape (84, 84, 1). This reduces input size and focuses on the motion. We already reviewed what a PPO agent represents, and the action is what that agent sends back to the environment as an output. Environment step represents an advancement in frames; in this case, it uses frame skipping, which we will review later in detail. Rollout buffer stores collected data from interactions after a certain number of steps. PPO policy update, which uses the policy data stored in

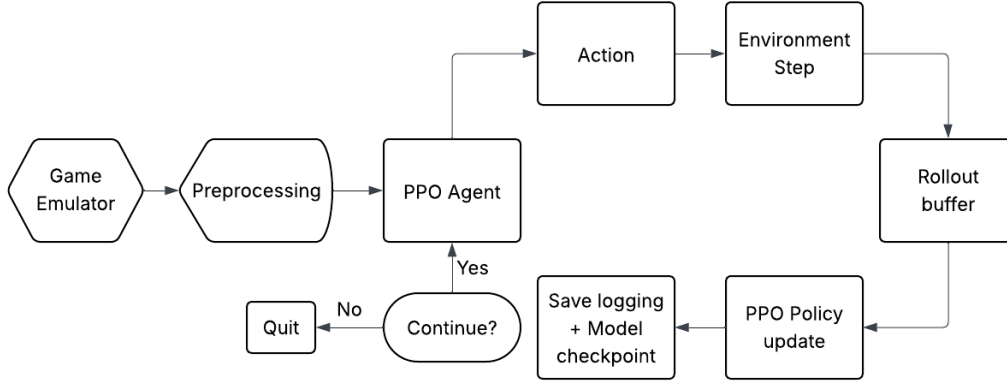


Figure 4.1: Visualization of the interaction between game environment and PPO agent

the buffer to calculate to update itself. Lastly, save login plus model checkpoints, which, as the name tells, represent loggings saved at regular intervals every 10000 steps. These logs can be used to update TensorBoard to track the progress. If the model does not reach the maximum number of steps, these actions will loop, or else it will stop.



Figure 4.2: Retro Gym Preprocessing Pipepile

Figure 4.2 illustrates the preprocessing steps applied to raw game frames before they are fed to the PPO agent. The pipeline consists of:

- **Grayscale Conversion:** RGB frames are converted to single-channel (84x84) to reduce computational complexity.
- **Resizing:** Frames are downsampled to 84x84 pixels to standardize input dimensions.
- **Frame Stacking:** Four consecutive frames are stacked to capture temporal motion (e.g., attack animations).
- **Frame Differencing:** The agent observes *changes* between frames (current frame – previous frame) to highlight dynamic elements (e.g., character movements, attacks).

This pipeline ensures the agent receives compact yet informative observations, balancing computational efficiency with gameplay context.

4.2 Environment Setup

The training environment for the reinforcement learning agent is built around the classic fighting game Street Fighter II, accessed through OpenAI Gym Retro. Thanks to this framework's standardized way of interacting with the game, the agent can both observe what's happening on screen and send control commands to play.

Raw pixel data or RAM states, which represent the game's current state, are observations provided by the environment. These observations are sent to the agent at each time step. The agent responds by sending unique actions, such as moving, attacking, blocking, and jumping, which correspond to the game controls.

Since fighting games have tons of possible inputs and combos, the action space is simplified into a smaller set of meaningful moves. This includes different types of attacks and basic movements. To make learning easier, complicated combos that normally require pressing multiple buttons at once are broken down into simpler, one-step actions whenever possible.

Frame skipping, where each selected action is repeated for a predetermined number of frames before the agent chooses a new action, is used to increase computational efficiency and learning stability. As a result, the agent can concentrate on significant state transitions and make decisions less often.

The agent learns to behave the way we want by using reward shaping. It gets rewards based on things like winning rounds, how much damage it deals to the opponent, and how much damage it takes. This structured feedback helps the agent learn faster and better understand the main goals of the game.

The following Python code shows the custom Gym environment wrapper used to preprocess frames, define the action and observation spaces, and integrate with the Gym Retro emulator:

```
1 class StreetFighter(Env):
2     def __init__(self):
3         super().__init__()
4         self.observation_space = Box(low=0, high=255, shape=(84,
5             84, 1), dtype=np.uint8)
6         self.action_space = MultiBinary(12)
7         self.game = retro.make(game='
8             StreetFighterIISpecialChampionEdition-Genesis',
9                 use_restricted_actions=retro.
10                     Actions.FILTERED)
11
12     def reset(self):
13         obs = self.game.reset()
14         obs = self.preprocess(obs)
15         self.previous_frame = obs
```

```

12         self.score = 0
13         return obs
14     def preprocess(self, observation):
15         gray = cv2.cvtColor(observation, cv2.COLOR_BGR2GRAY)
16         resize = cv2.resize(gray, (84, 84), interpolation=cv2.
            INTER_CUBIC)
17         channels = np.reshape(resize, (84, 84, 1))
18         return channels
19     def step(self, action):
20         obs, reward, done, info = self.game.step(action)
21         obs = self.preprocess(obs)
22         frame_delta = obs - self.previous_frame
23         self.previous_frame = obs
24         reward = info['score'] - self.score
25         self.score = info['score']
26         return frame_delta, reward, done, info
27     def render(self, *args, **kwargs):
28         self.game.render()
29     def close(self):
30         self.game.close()

```

Listing 4.1: Custom Gym environment for Street Fighter II using Gym Retro

This wrapper enables simplified state representation through grayscale and re-sizing, while the ‘MultiBinary(12)’ action space allows the agent to press multiple buttons simultaneously. The use of frame difference (‘frame_delta’) helps the model detect movement more effectively, which is critical in high-speed fighting games.

4.2.1 Frame Skipping and Reward Shaping

To make training more efficient and less demanding on computer power, the environment uses frame skipping. Instead of picking a new action every single frame, the agent repeats the same action for a fixed number of frames, which is usually about every 4 frames. This reduces the agent’s need to make decisions often, which lets it focus on bigger, more important changes in the game. Frame skipping also helps smooth out small frame-to-frame differences that do not really affect gameplay, making the training more stable overall.

The agent also benefits from reward shaping, which gives more frequent and useful feedback than just winning or losing rounds. Instead of only getting rewards at the end, the agent earns points for dealing damage to the opponent, encouraging it to be aggressive, and loses points when it takes damage, encouraging defense. Additionally, winning a round gives a big reward, while losing results in a penalty. This layered reward system helps the agent understand combat better and learn faster.

By combining frame skipping with reward shaping, the training setup strikes a good balance between saving computational resources and giving the agent clear guidance. This way, the reinforcement learning agent can better handle the challenges of Street Fighter II and improve more effectively.

4.3 Reinforcement Learning Algorithm

4.3.1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a top-tier reinforcement learning algorithm that strikes a great balance between sample efficiency and training stability, perfect for complex, high-dimensional environments like fighting games. PPO falls within policy gradient methods and uses an actor-critic framework: the actor learns which actions to take, and the critic estimates expected returns.

A core innovation in PPO is its clipped surrogate objective, which prevents large, destabilizing updates. The objective is:

$$L^{\text{CLIP}}(\theta) = \hat{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the new and old policy.
- $\hat{A}_t$ is the advantage estimate at time t .
- ϵ is a small positive hyperparameter (commonly 0.1 or 0.2) that controls the clipping range.

This objective encourages the new policy to improve expected advantages while staying close to the old policy, which promotes stable and reliable learning. PPO also supports multiple epochs of mini-batch updates using collected trajectories, improving data efficiency. [Schulman et al. \(2017\)](#)

In the context of Street Fighter II, PPOs ability to adapt continuously and manage complex action spaces makes it a strong candidate for training agents that can learn intricate fighting strategies and adapt to varied opponents.

4.3.2 Model Architecture

As I already mentioned, the RL agent is built using a Proximal Policy Optimization algorithm with a Convolutional Neural Network (CNN) policy, provided by the Stable-Baselines3 library. The architecture is designed to process visual input from the game Street Fighter II played through the gym-retro interface.

The visual input to the model comes in the form of grayscale game frames. Each frame is first converted to grayscale, resized to 84×84 pixels, and treated as a single-channel image. In order to capture temporal dynamics, such as movement, combos, or attacks, that develop over time, the model is trained using a stack of the previous four frames. These stacked frames result in an input shape of $(84, 84, 4)$, effectively giving the agent a short "memory" of what just happened.

However, during interaction with the environment, the custom game environment only produces one frame at a time. This is actually intentional since the environment is set up to produce a single $84 \times 84 \times 1$ preprocessed frame. Using Stable-Baselines3's VecFrameStack wrapper, which collects and stacks the last four frames during training, the frame stacking is handled separately. This design separates the responsibilities: the environment stays lightweight and fast, while the wrapper takes care of adding temporal context when needed. It also makes the setup more flexible and compatible with different reinforcement learning algorithms.

The CNN itself is based on the Deep Q-Network paper's architecture. A ReLU activation function is used to introduce non-linearity after each of the three convolutional layers, which have decreasing kernel sizes and increasing filter depth. From edges and motion to more complex patterns like hit animations or character spacing, these layers progressively extract higher-level features from the image.

After the convolutional layers, the output is flattened and passed through a fully connected layer with 512 units. This number "512" is a standard design choice from the Stable-Baselines3 library's default CNN policy architecture, which balances model complexity and performance for visual reinforcement learning tasks. This fully connected layer serves as a shared feature representation for two separate output "heads." One head is the policy network, which determines the probability distribution over the available button combinations (actions). The other is the value network, which calculates the anticipated cumulative reward from the current scenario.

An additional technique called frame differencing is also applied. Instead of learning from the raw frame, the agent learns from the difference between the current and previous frames. This removes static background noise and highlights motion, such as when a character moves or attacks.

Hyperparameters like learning rate, discount factor, and PPO clipping range are optimized automatically using Optuna. The model is trained using vectorized environments and regular checkpoints to monitor performance and save the best-performing versions.

4.4 Training Procedure

The goal of the reinforcement learning agent’s training process is to gradually improve the agent’s Street Fighter II fighting skills through a continuous cycle of interaction, learning, and evaluation. Setting up the environment and the PPO agent is the first step in the process. This is followed by multiple episodes where the agent observes the game state, behaves, gets feedback, and updates its policy to maximize cumulative rewards.

Observations are provided by the environment as 84×84 pixel preprocessed grayscale frames. Four consecutive frames are stacked in order to capture the temporal dynamics that are important in fighting games. This allows the agent to detect motion and hidden gameplay cues like combos and attacks. Vectorized wrappers effectively manage this frame stacking outside of the environment, ensuring a modular and effective configuration.

Before training starts, the PPO agent is instantiated with hyperparameters selected through automated tuning. These hyperparameters regulate critical aspects of the learning process, including the number of steps between policy updates, the discount factor for future rewards, the learning rate, and the clipping parameter which regulates how much policy updates can differ from earlier iterations, are all important aspects of the learning process that are controlled by these hyperparameters.

By pre-optimizing the hyperparameters, the training process is made more reliable and efficient. The agent’s model weights are initialized from a previously saved checkpoint rather than starting from scratch during training. Instead of relearning basic behaviors, this transfer learning approach makes use of existing knowledge to provide a solid foundation that allows the agent to keep improving its strategies.

The agent repeatedly interacts with the environment during training by choosing actions based on its current policy. Each action corresponds to pressing or releasing a combination of buttons relevant to the fighting game. The environment provides a reward signal based on score changes, a new observation, and information about whether the game round has ended after an action is completed. The agent is encouraged to use efficient offensive and defensive strategies by the reward design, which takes into account damage both given and received.

The PPO algorithm then updates policies based on the experience that has been gathered. It calculates advantage estimates, or how much better certain actions were compared to average expectations, and modifies the policy in accordance with those estimations. By preventing huge policy changes that might destabilize learning, the clipped surrogate objective function allows the agent to gradually and accurately improve performance over time.

The environment uses frame skipping, which repeats each selected action for a number of frames before choosing a new one, to control computational resources and improve training stability. Focusing on important state transitions reduces the frequency of decisions and minimizes the learning signal.

4.4.1 Data Collection (Rollouts)

As the agent begins by playing episodes of *Street Fighter II* within the Gym Retro environment, at each timestep, it selects an action based on the current policy and receives an observation, reward, and termination signal from the environment. This process is repeated for a fixed number of steps, in this case 2048, after which the collected trajectories (state, action, reward, next state, done) are used for training.

4.4.2 Policy and Value Update

Once a batch of rollouts is collected, the PPO update is performed. The update intends to minimize the policy’s clipped surrogate loss while also lowering the mean-squared error between the estimates of the actual and predicted values. This maintains learning stability by preventing the policy from changing significantly in a single update.

4.4.3 Hyperparameter Configuration

To optimize learning performance, hyperparameters were tuned using Optuna, a gradient-free optimization framework. The search was conducted over multiple trials with PPO as implemented in the Stable-Baselines3 library. The final configuration was selected based on the best-performing trial after evaluating on held-out episodes.

Table 4.1: Key Hyperparameters Used During PPO Training

Hyperparameter	Value / Range
Learning Rate	5.2×10^{-5} (tuned in $[1 \times 10^{-5}, 1 \times 10^{-4}]$)
Discount Factor (γ)	0.987 (tuned in $[0.8, 0.9999]$)
GAE Parameter (λ)	0.927 (tuned in $[0.8, 0.99]$)
Clip Range	0.234 (tuned in $[0.1, 0.4]$)
Number of Steps (Rollout Size)	7424 (searched in $[2048, 8192]$)
Frame Stack	4 (to capture temporal context)
Batch Size	64256 (depends on rollout division)
Epochs per Update	4 (PPO default)
Entropy Coefficient	0.00.01 (default range, not explicitly tuned)
Optimizer	Adam (default momentum and ϵ values)
Frame Skip	4 (action repeated every 4 frames)

These hyperparameters were selected based on empirical performance during training at both 3 and 7 million timesteps. The use of Optuna allowed automated exploration of the high-dimensional hyperparameter space, leading to more sample-efficient training and improved policy convergence.

4.4.4 Stability Considerations

Multiple kinds of techniques are used to improve learning stability and avoid issues like overfitting or unpredictable behavior in order to provide smooth and reliable training. First, reward normalization makes learning easier by maintaining reward values across episodes. To avoid sudden, significant updates that might cause the policy to become unstable, gradient clipping is applied. We can track progress and stop training if the agent’s performance drops, thanks to early stopping and ongoing evaluations. When combined, these techniques allow the agent to learn efficiently without shifting from or getting stuck in poor strategies.

4.4.5 Hardware and Runtime

The training was performed on a laptop with an Intel i7-10750H CPU (12 cores, up to 5.0 GHz) and an NVIDIA GeForce RTX 2070 Mobile GPU (Max-Q Refresh) with 32 GB of RAM. The operating system used was Fedora Linux.

The initial hyperparameter tuning phase took approximately 2 days to complete, as it involved 10 PPO trials exploring different configurations. Subsequent training with the best-found parameters ran for around 8 hours.

Although this setup is not ideal for model training, particularly since it is a laptop, it still can provide results sufficient to explore and develop broader theoretical ideas.

4.5 Evaluation Strategy

To assess the performance and behavioral quality of the reinforcement learning agent in *Street Fighter II*, both quantitative and qualitative evaluation methods are used. In addition to raw performance metrics like win rate, these evaluations look to capture the agent’s gameplay’s strategic depth and adaptability. This extensive evaluation provides insights into the agent’s ability to engage in dynamic fights.

- **Win Rate:** The agent’s win rate is measured by playing a fixed number of evaluation episodes against a hard-coded, static opponent. To ensure consistency across tests, this opponent relies on either random actions or a simple scripted policy. The win rate, which is the proportion of matches that the agent wins, is a clear indicator of the agent’s ability to defeat enemies.
- **Average Episode Reward:** This metric tracks the mean cumulative reward obtained by the agent across evaluation episodes. Since rewards are derived from game-specific metrics such as score differences or in-game events, this value reflects the agent’s ability to optimize the reward function defined during training. A higher average reward typically corresponds to better in-game performance.
- **Action Diversity:** The agent’s diversity of actions is observed to prevent simplistic or repetitive strategies. This includes counting the number of unique actions or action sequences which are used during gameplay. A richer, more human-like playstyle is suggested by high action diversity, which also shows that the agent is successfully navigating the action space rather than relying on just one strategy.
- **Opponent Behavior Response:** Observing how the agent responds to changes in the opponent’s strategy is the qualitative metric. For example, the agent’s ability to change its behavior, whether by retreating, blocking, or starting different combos, is noted when it encounters an aggressive or defensive opponent. In the context of player-specific learning, these observations show whether the agent has acquired any kind of pattern recognition or strategic adaptivity.

Together, these metrics provide a comprehensive evaluation framework. Quantitative results give objective insights into training effectiveness, while qualitative analysis helps uncover more subtle aspects of gameplay such as adaptivity, creativity, and behavioral realism.

4.5.1 Baseline Comparison

The performance of the reinforcement learning agent should be compared to a pre-determined baseline agent in order to be meaningfully evaluated. This baseline serves as a point of reference to put the agent’s capabilities in context and represent various levels of strategic complexity. By establishing lower and upper bounds of performance, the comparisons reveal how well the learned policy performs beyond hard-coded strategies.

The built-in AI from *Street Fighter II* is designed by game developers to provide engaging gameplay for human players and includes reactive behavior, timing-based attacks, and basic pattern recognition. It serves as a strong, consistent baseline to test how well the reinforcement learning agent generalizes to real gameplay scenarios.

By comparing the trained agent against this baseline across metrics such as win rate, reward accumulation, and action diversity, we can evaluate the relative efficiency of the learning process. While competing with or surpassing the in-game AI indicates that the model has learned profitable behaviors, it also refers to the emergence of complex, competitive strategies.

4.6 Player-Specific Adaptation

While the model I trained is about a general-purpose reinforcement learning agent capable of playing Street Fighter II, the main goal is to have an agent designed to adapt to player-specific gameplay. In real-world scenarios, especially in games designed for human interaction, an intelligent agent that can recognize and adjust to a specific player’s behavior is one of the biggest steps not only in fighting games but generally in machine learning. It holds substantial potential for improving engagement, challenge balancing, and immersion.

4.6.1 Motivation

We already mentioned how modern fighting games still rely on static or hard-coded AI opponents, which can become predictable and fail to adapt to humans’ evolving strategies. A real player-specific opponent would observe and adapt to a player’s behavior, such as favored combos, attack frequency, or defensive habits. This type

of personalized gameplay feels more dynamic and tailored, presenting a greater challenge and improved replayability.

The long-term vision for this line of work is to build an AI that:

- Learns to recognize recurring patterns in a players actions, such as aggressive rushdowns, zoning, and passive defense.
- Adapts its strategy over time to counteract and challenge the specific opponent.
- Continually evolves through extended play sessions with the same user.

4.6.2 Limitations

Although the primary motivation behind this project is the idea of player-specific adaptation, it is still only theoretical at this point. No complete player modeling implementation or prototype has been created due to practical limitations. Instead, the approach is explored through discussion and early design considerations, creating an environment for further research. The main reasons why player-specific adaptation could not be fully pursued in this project are highlighted by the following limitations:

- **No real player data:** The agent has not been trained or fine-tuned using real human gameplay logs due to time and resource limitations. Its responses are therefore only measured in comparison to artificial opponents.
- **No explicit opponent modeling:** The agent does not use a separate module to identify patterns in behavior over time, nor does it keep an internal model of its opponent. All adaptation, if any, is only implied through policy generalization.
- **No online or continual learning:** The agent does not update its policy during gameplay or across episodes based on a specific opponent. All learning is done offline during the standard training phase.
- **Computational limitations:** True player-specific adaptation often requires fine-tuning or meta-learning strategies, which are computationally intensive and generally require more powerful hardware than what is available in this project (a gaming laptop with limited training time).

4.6.3 Significance and Future Work

Although the agent has not yet demonstrated major, focused adaptation, this prototype is a significant step toward developing adaptive game AI. The experiments provide a baseline understanding of how the current PPO agent operates under opponent change and provide insight into whether general RL policies fundamentally encode some level of responsiveness.

This foundation can be developed in future projects by:

- Integrating online learning tools that enable the agent to modify its strategy in the middle of a game.
- Gathering actual player data in order to train or improve on specific opponent profiles.
- Exploring opponent modeling or meta-learning strategies.
- Increasing simulation time and utilizing more powerful hardware to scale training in order to handle more complex adaptation strategies.

In conclusion, this project section shows the early exploration of player-specific adaptation in AI fighting games. Even though its scope is limited, it provides proof that future extensions are achievable and could make AI opponents more intelligent, difficult, and responsive to players in a unique way.

Chapter 5 Results and Discussions

5.1 Overview

This chapter presents and analyzes the experimental results obtained from training a reinforcement learning agent based on the Proximal Policy Optimization (PPO) algorithm to play *Street Fighter II*. Understanding the agent’s behavior, performance, and learning curve over time is the main goal of the evaluation. The agent’s ability to compete in the fighting game environment is evaluated using both quantitative metrics and qualitative gameplay observations.

The evaluation was conducted against the default AI built into *Street Fighter II*, which serves as a benchmark for comparison. Unlike the trained PPO agent, this AI follows predefined rules and does not learn or adapt, but still exhibits consistent and competent behavior designed by the original game developers.

We want to determine whether the learned policy outperforms hardcoded strategies and whether the PPO-trained agent can demonstrate some level of reactivity, strategy, or adaptability while playing by comparing it with this baseline.

This chapter also reflects on the development process by comparing two distinct implementations of the agent:

1. An **initial version** that performed noticeably better, successfully engaging with opponents and executing more strategic actions, including attacks, movement, and blocking.
2. A **revised and improved version** that is a retrained variation of the initial version, but this time its behavior has improved by standing up and throwing a punch, blocking, and actually engaging in combat.

This comparison between the two implementations offers insights into the sensitivity of reinforcement learning agents to changes in reward structure, environment setup, and training stability. It also illustrates how small implementation flaws can significantly affect the learning outcome.

5.2 Training Performance

The PPO agent was trained over multiple timesteps to learn effective fighting strategies in *Street Fighter II*. Two different training runs were conducted: an initial implementation trained for 3 million timesteps, and an improved version trained for 7 million timesteps. Both agents were evaluated over 10 trials each. Key metrics such as episodic reward, policy loss, and value loss were monitored during training to assess learning progress and convergence.

5.2.1 Episodic Reward Trend

Figures 5.1 and ?? show the moving average of episodic rewards across training episodes for the initial and improved agents, respectively.



Figure 5.1: Training performance metric for PPO agent at 3 million timesteps.

In the figure 5.1, we can see a huge downfall in the beginning. That's because the model was still very unfamiliar with the environment, but if you noticed, it gradually started to evolve. Even though this version of the agent is relatively more defensive than the other one, since it's always crouched, we can see that it still has the opportunity to improve. My guess to explain that behavior is that, when the

training began, it discovered that the crouching position was the safest with the particular model it was training against, therefore it got stuck with it for the rest of the training. We can even compare it to a very emotional human as well, since the behavior in both variations is the same. Both of them get scared to do anything; therefore, they try to stick with the defensive position. But the graph indicates that the model is gaining confidence and is trying to engage more in a fight, which results in more reward.

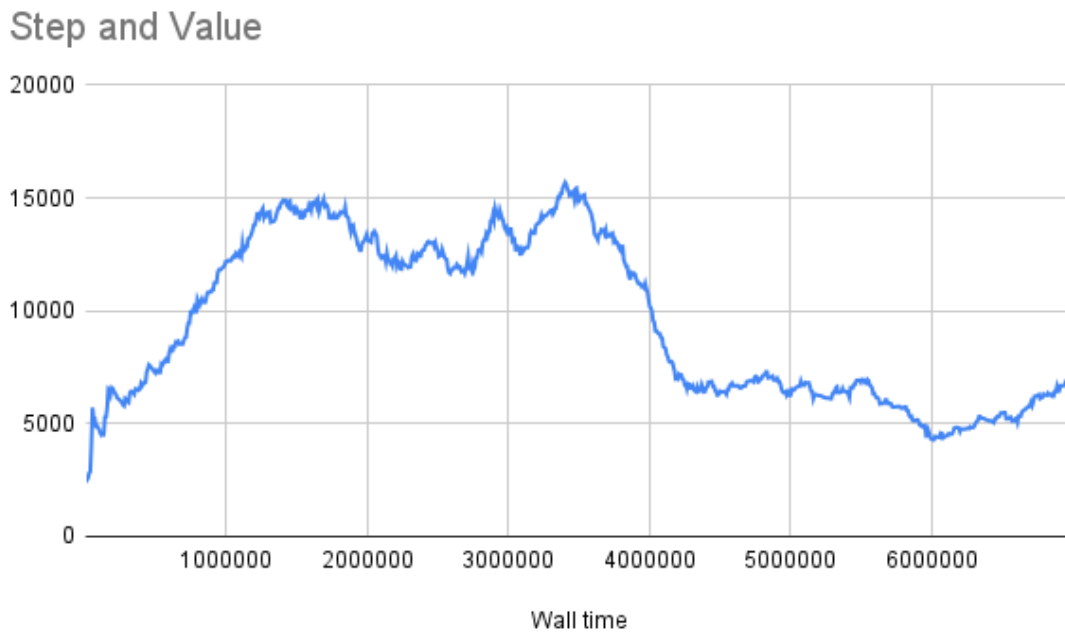


Figure 5.2: Training performance metric for PPO agent at 7 million timesteps.

On the other hand, we have figure 5.2, which is an advanced version of the first implementation. As we already mentioned, this model was trained on 7 million time steps, and we can see that it even gave us better results, even though it had a downfall after 3.5 mils. This version of the model actually beat the first integrated model the game offered. The previous agent was able to win some of the rounds, but mostly because of luck, but on this one, you can clearly see the calculated moves it makes. For example, it discovered blocking attacks, and sometimes it even does it perfectly. It also learned a few complex combos. All of them are done by leg, since the first implementation was only using leg, it just made leg its main weapon. Therefore, this model was able to advance to the next round with another opponent. And that's when the downfall starts. The methods that worked with the first opponent didn't work with the second one anymore. That's why it kept losing rewards and didn't know what to do. But if you notice, after 6 million timesteps, there is an incline start. If I had let it train more, it would probably be able to overcome both first and second opponents.

5.2.2 Policy Loss and Value Loss

Both agents showed stabilization in policy and value loss over time, indicating convergence of their learning processes. The improved agent exhibited smoother loss curves with less fluctuation, reflecting the benefits of extended training. Minor oscillations are expected in PPO training due to its on-policy updates and continued exploration of the action space.

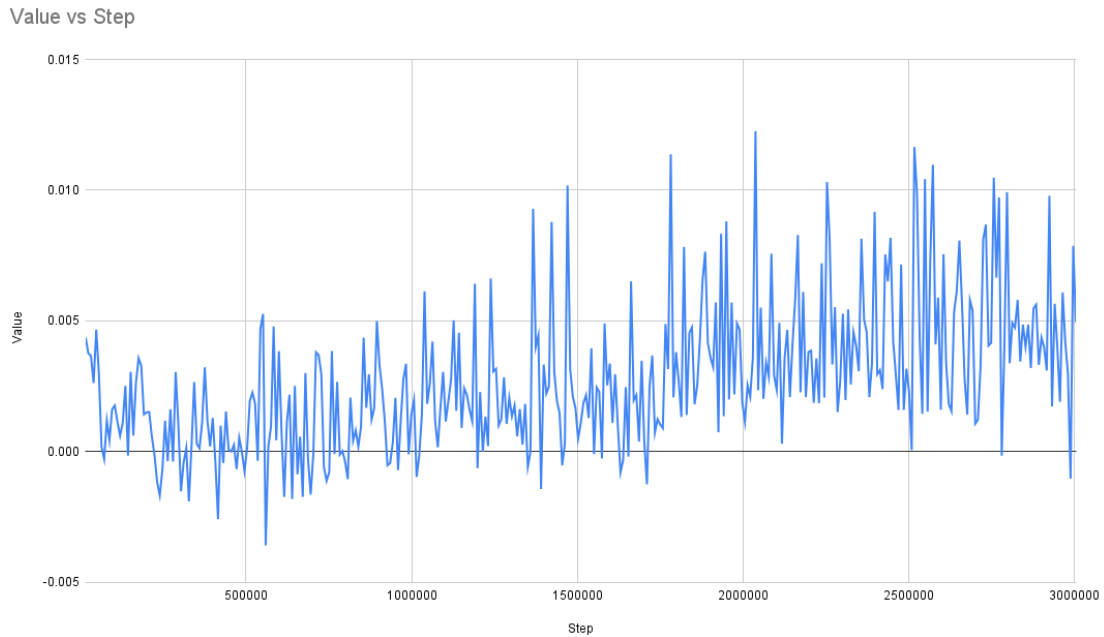


Figure 5.3: 3 million steps: Policy Loss

Figure 5.3 shows the policy loss after 3 million timesteps. As the model explores the action space, initial changes are expected; however, the overall trend should ideally stabilize, reflecting a shift toward an effective policy. Although there is some noise in this case, the loss is still bounded close to zero, which indicates that the model is beginning to learn stable actions.

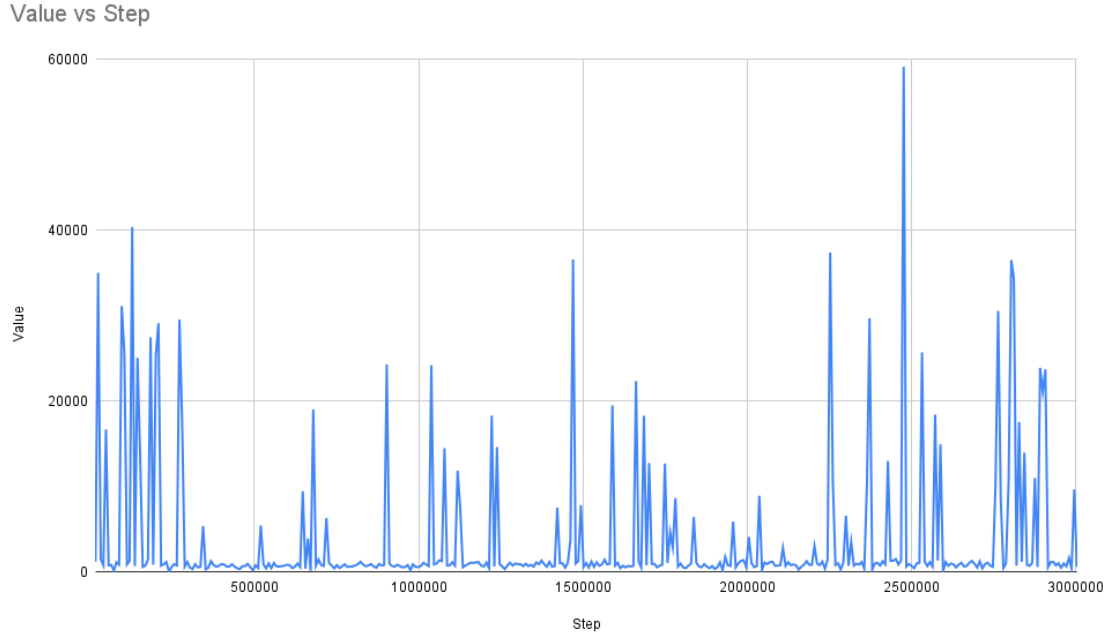


Figure 5.4: 3 million steps: Value Loss

Figure 5.4 presents the corresponding value loss at the same point in training. Since value estimates are updated continuously, the value loss displays high-frequency fluctuations, which are typical in PPO. The overall values are low despite being a little noisy, indicating a reasonable approximation of the value function.

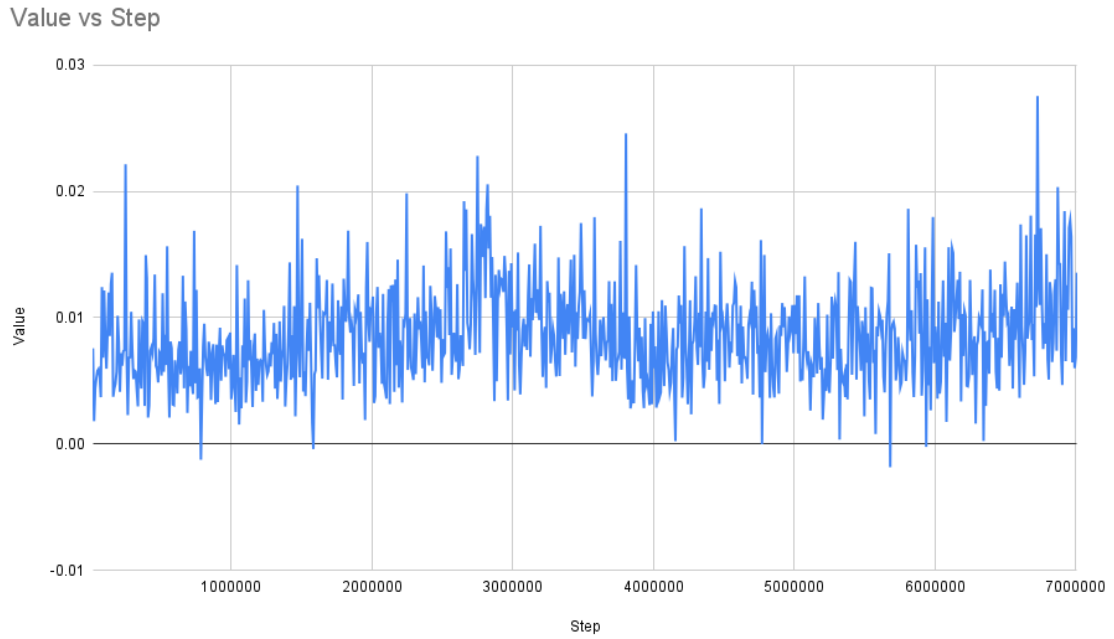


Figure 5.5: 7 million steps: Policy Loss

Figure 5.5 demonstrates the policy loss after 7 million timesteps. At this point,

policy loss is very low. It still fluctuates, but within a tighter bound, closer to zero than the first implementation, indicating that this model is more stable at making decisions and has likely settled into a more consistent strategy.

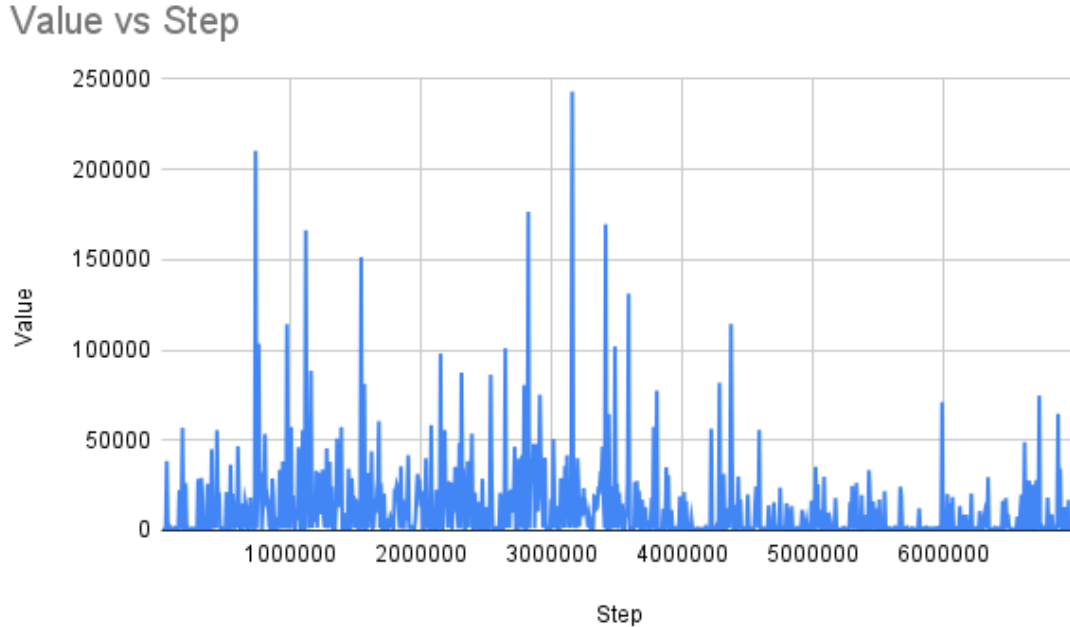


Figure 5.6: 7 million steps: Value Loss

Figure 5.6 displays the value loss after 7 million timesteps. As you can see, the model gains less value after 3.5 million. That’s because of the opponent change, which I described previously during the thesis. We can also see how sensitive the PPO agent can be on an opponent change since the value loss difference is drastic.

5.3 Quantitative Evaluation

5.3.1 Win Rate Against the Baseline

To evaluate performance improvements, both the initial and improved PPO agent implementations were tested against the built-in CPU opponent. In the first implementation, the agent occasionally managed to land attacks and even win a few individual rounds. However, it consistently failed to win full matches, often losing in the second or final round due to poor defense and inconsistent strategy.

In contrast, the second implementation, which benefited from extended training and hyperparameter tuning, demonstrated significantly better performance. The agent was often able to advance to the second round and often dominated the match with a mix of offensive combos and great blocking. This improvement in win rate

clearly reflects the advantages of deeper training and a more stable optimization process.

5.3.2 Action Diversity

The action patterns of the agents revealed notable differences. Before I started to train an agent, I first examined how the random agent would perform in the same environment. The Random Agent exhibited a wide range of actions, including various combos, punches, kicks, jumps, and blocks. However, these actions were often uncoordinated and lacked strategic timing, resulting in hits that were mostly out of place.

Implementation 1 showed limited action diversity, primarily being crouched and only throwing a lower kick, which is neither a great offensive nor defensive stance, but it did manage to land a few kicks, which caused a few wins in rounds, but it's still ultimately failed when it came to actually winning a match.

Implementation 2 demonstrated more offensive activity than Implementation 1, predominantly crouched agent was able to demonstrate some high skill combos. As I already mentioned, those combos were all leg-based since the crouching position offers more flexibility with the legs. It also occasionally stands up to throw a punch, which indicates that it's gaining confidence in the environment. But I think the most important aspect of this evolution of the freshly learned ability to block since this decreased the health loss and gave the agent opportunities to counterstrike.

Overall, the trained implementations showed less diversity in their actions compared to the Random Agent, indicating a tendency to rely on a narrow set of moves rather than exploring a broad action space.

5.4 Qualitative Observations

5.4.1 Combos and Reactive Behavior

The trained agents displayed varying levels of skill in executing combos and reacting to opponent actions. As we already mentioned, implementation 1 rarely executed any multi-hit combos, largely due to its tendency to crouch and avoid engaging aggressively. This defensive posture limited opportunities to chain attacks and capitalize on openings.

Implementation 2 showed some capacity to perform simple attacks, such as lower kicks and occasional punches, as well as multi-hit leg combos. Even though it performed some of the combos, there are still a vast variety of them to be learned. The lack of more complex sequences suggests either insufficient training time or

limitations in its policy to recognize and exploit combo opportunities.

The Random Agent, despite lacking a strategy, occasionally performed diverse combos by chance. However, these were often mistimed and not strategically advantageous, leading to lower overall efficiency.

Regarding reactive behavior, both trained implementations showed minimal adaptive responses. Implementation 2 sometimes reacted to opponent movements by adjusting its stance but it still need to be more engaging and confident to actually be able to defeat multiple opponents. Implementation 1, with its retreating behavior, effectively avoided damage but missed chances to punish opponents.

In contrast, the Random Agent exhibited no reactive behavior, acting purely on stochastic action selection without regard to the opponent’s moves.

These observations suggest that while the trained agents learned some basic attack patterns, their reactive capabilities and combo execution remain limited, indicating potential areas for improvement such as enhanced state representation or reward shaping focused on combo effectiveness and counterplay.

5.4.2 Limitations in Complex Scenarios

The trained agents exhibited notable weaknesses when confronted with complex or unpredictable situations. For example, when cornered or pressed against the edges of the stage, both implementations struggled to maintain effective offensive or defensive strategies. Implementation 1 frequently defaulted to crouching and executing low kicks without adapting its behavior to the changing context. This repetitive pattern made it predictable and exploitable by more varied or aggressive opponents.

Implementation 2, on the other hand, often did the same as the initial version, but it had more escape opportunities since its move set was broader. But overall, it still lacks lots of abilities.

Additionally, in scenarios involving unpredictable or highly variable opponent movement, both implementations showed limited exploration of alternative tactics, indicating a possible convergence to local optima during training. This lack of behavioral diversity increases the possibility that the agents’ policies lack the motivation or ability to generalize beyond the states that are most frequently encountered.

5.5 Opponent Behavior Sensitivity

The trained agents performance was notably influenced by the specific behavior patterns of their opponents. Both implementations displayed varying degrees of adaptability depending on the consistency and predictability of the opponents actions. Against opponents with repetitive or scripted patterns, the agents were able

to learn effective counters and exploit weaknesses, resulting in higher win rates and more strategic play.

However, when facing opponents with more stochastic or reactive behaviors, the agents struggled to maintain consistent performance. Implementation 1s tendency to rely on a limited set of defensive strategies made it less effective against opponents that could exploit its predictability. Implementation 2, while slightly more aggressive, showed reduced flexibility in adjusting tactics mid-match, often repeating the same attacks regardless of the opponents changing strategy.

These observations suggest that the agents are sensitive to the diversity and complexity of opponent behavior encountered during training and evaluation. The limited generalization to novel or highly variable opponent strategies points to potential areas for improvement, such as incorporating opponent modeling, meta-learning techniques, or enhanced exploration mechanisms to better adapt in real-time to different playstyles.

5.5.1 Toward Player-Specific Adaptation

A key goal of this project is to move beyond static AI behavior and develop agents capable of adapting to individual player styles. While the current implementations demonstrate basic strategic learning, they lack mechanisms to personalize their tactics based on the specific opponents tendencies observed during gameplay.

Player-specific adaptation would involve dynamically adjusting the agents policy to exploit habitual patterns, preferred moves, and timing preferences unique to each player. This requires real-time data collection and analysis of the opponents actions, followed by continuous policy refinement or switching between pre-trained sub-policies tailored to different playstyles.

Incorporating such adaptive capabilities promises a more challenging and engaging experience for human players, as the AI could counter their strategies more effectively over time. Future work should explore reinforcement learning techniques that support online learning, meta-learning frameworks, or hierarchical models that enable quick adaptation with minimal data.

This direction aligns with the broader objective of creating AI opponents that feel intelligent and responsive, enhancing the overall depth and replayability of fighting games.

5.6 Discussion

5.6.1 Strengths of PPO in Fighting Games

In the context of fighting games, the Proximal Policy Optimization algorithm has proven effective for training reinforcement learning agents. By avoiding excessively aggressive policy updates that could destabilize learning, its clipped objective function helped maintain training stability. In comparison to other RL algorithms, PPO was also able to learn strategies that were both effective and reasonably generalizable in a fairly short number of training steps. In terms of consistent behavior and basic offensive and defensive patterns, the trained agents' gameplay quality surpassed that of baseline agents such as Random and Scripted agents.

5.6.2 Challenges Faced

Despite the algorithm's strengths, several challenges emerged during implementation. Fighting games involve real-time, high-frequency interactions with complex visual feedback, which introduces significant variance in the agents decision-making. This variance made it difficult for the policy to converge without fine-tuning hyperparameters. Additionally, the problem of balancing exploration and exploitation was especially pronounced; aggressive exploration sometimes resulted in incoherent actions, while overly exploitative strategies tended to converge to suboptimal local policies. Another issue was the sparsity and delay of rewards associated with strategic behaviors such as blocking, spacing, or countering. These nuanced behaviors are crucial in high-level play but were rarely reinforced adequately under a standard PPO reward structure.

5.6.3 Limitations of This Implementation

The performance and scope of this PPO implementation in the fighting game domain were limited by a number of issues. The lack of diversity in the opponents' training was one of the main problems. The learning process was probably tilted toward taking advantage of particular patterns rather than learning generalized skills because the majority of training episodes were against static or limited behavior agents. Additionally, the agent did not have access to real-time data from human players, which would have supported more meaningful policy evolution and dynamic adaptation to human strategies. Furthermore, the depth and endurance of the learned behavior were ultimately limited by hardware limitations, such as limited GPU memory and CPU processing power, which also limited training batch sizes and the total number of training episodes.

5.7 Summary

This chapter examined how the PPO-trained agents behaved and performed in a fast-paced fighting game environment. Along with the observable gameplay improvements over baseline agents, the PPO algorithm’s benefits were highlighted, especially its stability and ability to learn efficient policies. However, lots of challenges came up, such as sparse strategic rewards, challenges in establishing a balance between exploration and exploitation, and high variance caused by real-time interactions. The current implementation’s flaws, including its lack of human-driven data, limited opponent diversity, and hardware limitations which affect the training scale, were also covered. These insights lay the groundwork for understanding the potential advantages and real-world limitations of integrating reinforcement learning into competitive, dynamic games.

Chapter 6 Conclusions

6.1 Summary of Work

This thesis researched the use of reinforcement learning, particularly the Proximal Policy Optimization algorithm, to build adaptive agents in the 2D fighting game Street Fighter II. We also reviewed other applications of RL to different games and approaches, which gave us deeper knowledge in this field. The observation that traditional game AIs, like scripted or rule-based agents, frequently lack the ability to adapt dynamically to human players or display a variety of strategic behaviors worked as the motivation. The objective was to develop agents that could learn complex strategies through self-play and be evaluated against fixed baselines by using PPO.

By presenting significant concepts like Markov Decision Processes, policy optimization, and popular RL algorithms like DQN, A2C, SAC, and PPO, the early chapters formed a solid foundation in RL fundamentals. The challenges in applying these algorithms in fighting games, such as a requirement for opponent modeling, complex action spaces, sparse rewards, and real-time decision-making, were highlighted.

The literature review highlighted major advances in game AI, from classical scripting to deep RL approaches, and detailed how large-scale systems like OpenAI Five and AlphaStar paved the way for learning-based strategies in complex environments. FightingICE and related platforms were given special attention as research testbeds for the creation of AI fighting games.

The implementation phase used the Retro Gym environment, based on classic fighting games, to train and evaluate two PPO-based agents. These agents were designed with different training setups and hyperparameters to test how architecture and reward shaping influence learning outcomes. Training procedures included roll-out generation, policy updates, reward normalization, and optimization constraints such as clipping to ensure PPO's stability.

Both quantitative and qualitative methods were used for the evaluation. The agents were assessed quantitatively using action frequency distributions, loss met-

rics over time, and win rates compared to baseline agents (both scripted and random). Qualitatively, recorded matches were used to examine gameplay behaviors like combo execution, reactive blocking, movement patterns, and situational awareness.

The thesis’s final chapter covered the latest developments, PPO’s advantages, and a number of implementation-specific limitations, including a lack of real player data and limited opponent diversity. A roadmap for further research was presented, with a focus on curriculum learning, more scalable training architectures, and real-time player modeling.

All things considered, this work provides useful research on how PPO can be applied successfully in fast-paced, competitive games and creates a clear route to develop intelligent, adaptive opponents that can pick up on and react to a variety of player strategies.

6.2 Key Findings

Several key findings that strengthen our knowledge of reinforcement learning in dynamic, real-time games have been discovered from the implementation, training, and evaluation of PPO-based agents in a competitive fighting game environment:

1. PPO Enables Stable and Effective Policy Learning: One promising method for handling the challenges of fighting games was Proximal Policy Optimization. In many cases, PPO consistently created agents that performed better than baseline scripted opponents, even with the high variance in real-time environments and noisy observations. By avoiding performance regressions frequently brought on by major changes in policy, its clipped objective function helped in maintaining learning stability.

2. Emergent Behavior Without Explicit Supervision: Despite lacking any specific training in fighting game strategy, the agents developed noticeable behavioral patterns, such as crouching attacks, retreating under pressure, or spacing. Particularly the reward structure and iterative learning from interaction shaped these emergent strategies.

3. Baseline Comparisons Reveal Nuanced Agent Weaknesses: The win rate did not always indicate a better strategy, even though PPO-trained agents performed better than scripted agents in certain trials. PPO agents lacked sufficient offensive initiative and combo chaining, as proven by the fact that random agents with chaotic but high-volume action output occasionally outperformed trained models in terms of win rates. This brought to light an important trade-off between raw aggression and learned discipline.

4. Action Diversity Was Often Limited: Trained agents preferred safe, repetitive actions that offered short-term rewards even when a large action space was available. Their limited action diversity made it harder for them to adjust to various opponents and made them less successful in certain environments.

5. PPO Struggles Without Rich Opponent Variety: The agent’s poor performance during training as a result of the lack of diversity in opponents was one of the most obvious issues. The agent became overly specialized, adjusting its strategy to fit those fixed patterns because it only had to deal with a small number of hardcoded behaviors. Therefore, even minor changes in the opponent’s strategy, like a different attack rhythm or more aggressive spacing, could cause the agent to perform differently. This demonstrates the importance of opponent diversity in seeking more robust and generalizable fighting game behaviors.

6. Reward Shaping Is Critical Yet Difficult: The developed strategies were heavily influenced by the structure of the rewards. The behavior of the agent would frequently change significantly even if only minor adjustments were made to how it was rewarded or penalized. This was especially true for complex tactics that provide long-term benefits but don’t produce instant benefits, like blocking, countering, or baiting. In the absence of careful reward shaping, the agent frequently ignored hidden yet powerful strategies in exchange for simpler, more straightforward ones.

7. Potential for Player-Specific Adaptation Exists: Although the scope of this project didnt include real-time learning from human players, some early findings were encouraging. The agent gradually started to adapt and take advantage of predictable flaws after being repeatedly exposed to a single scripted opponent with a unique fighting style. This behavior is far from true adaptation, but it indicates that PPO-based agents may be moved toward player-specific learning in future research with more training structures and access to human gameplay data.

6.3 Limitations

We already mentioned and discussed what limited this project from being successfully implemented player-specific agent. The following reasons were:

- **Limited Opponent Diversity:** Most training episodes involved scripted or hardcoded agents with predictable behavior, which led to overfitting and reduced generalization.
- **No Real-Time Human Data:** The agent was not exposed to gameplay data from real players, limiting its ability to adapt to dynamic, human-like strategies.

- **Hardware Constraints:** Limited GPU memory and CPU performance restrict training batch sizes, model complexity, and total training duration, hindering the agents ability to learn long-term strategies or deeper patterns.
- **No Live Tracking:** The lack of internal state diagnostics and real-time player behavior tracking in the system prevented real-time adaptation or evaluation as well as complex debugging and behavior analysis.
- **Sparse Reward Challenges:** Because delayed or limited rewards made it difficult to reinforce strategic actions like blocking, countering, or spacing, complex tactics were discouraged in favor of simpler attacks.
- **Simplified Training Environment:** The realism and complexity of the trained agent’s behavior were limited by the action space and observation model’s constraints for quicker prototyping.
- **Prototype-Only Adaptation Design:** While the framework for player-specific learning was outlined, the actual adaptation mechanisms remained unimplemented due to time constraints.

6.4 Future Work

This project lays the groundwork for player-specific adaptation in fighting game AI, but full implementation remains theoretical. The current PPO agent is general-purpose and does not learn from or adapt to individual player behaviors. Key limitations include the lack of real human gameplay data, the absence of explicit opponent modeling, no support for online or continual learning, and hardware constraints. Despite this, the early results suggest the potential for adaptive behavior through reinforcement learning.

Future work should focus on integrating real-time learning mechanisms, collecting and leveraging player-specific data, implementing opponent modeling strategies, and scaling training with more powerful hardware. All of this may eventually result in AI opponents that are more dynamic, intelligent, and appealing and that gradually adjust their tactics to suit each player.

6.5 Final Remarks

In the environment of the classic fighting game Street Fighter II, this thesis researched the use of reinforcement learning, particularly the Proximal Policy Optimization algorithm. While the agent played well against opponents that were

hardcoded, the main goal of player-specific adaptation is still a challenge for further research.

This thesis provides a useful starting point for understanding PPO’s advantages and disadvantages in a dynamic, real-time environment. It also highlights the particular challenges in including dynamic, human-like learning in-game AI, difficulties that extend beyond technical implementation and consider careful environment design, more extensive data, and continuous improvement.

This project is a significant step toward more intelligent, responsive, and adaptive game agents, despite time and hardware limitations. AI may be able to understand and communicate with human players more effectively thanks to reinforcement learning, which could improve gameplay quality and immersion with further study and advancement.

As a proof-of-concept, this work ultimately encourages future researchers to build upon these concepts, improve the strategies, and advance the field of adaptive game AI.

Bibliography

- Berner, C., Brockman, G., Chan, B. et al. (2019), ‘Dota 2 with large scale deep reinforcement learning’, *arXiv preprint arXiv:1912.06680* .
- Chess.com (1997), ‘Deep blue vs. garry kasparov: The match that made history’.
URL: <https://www.chess.com/article/view/deep-blue-kasparov-chess>
- Datapod, S. (2023), ‘The history of reinforcement learning’.
URL: <https://researchdatapod.com/history-reinforcement-learning/>
- Haarnoja, T., Zhou, A., Abbeel, P. & Levine, S. (2018), ‘Soft actorcritic: Offpolicy maximum entropy deep reinforcement learning with a stochastic actor’.
- Ikeda, K. & Arita, T. (2018), Reinforcement learning approach for adaptive game ai with opponent modeling, *in* ‘Proceedings of the 14th AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment’.
- Isla, D. (2005), Handling complexity in the Halo 2 AI, *in* ‘Proceedings of the Game Developers Conference (GDC)’. Recording.
URL: <https://gdcvault.com/play/1020270/Managing-Complexity-in-the-Halo>
- Luu, Q. T. (2025), ‘Q-learning vs. deep q-learning vs. deep q-network’, *Baeldung* .
URL: <https://www.baeldung.com/cs/q-learning-vs-deep-q-learning-vs-deep-q-network>
- Matsumoto, S., Ito, K. & Togelius, J. (2018), Fighting game ai competition platform, *in* ‘2018 IEEE Conference on Computational Intelligence and Games (CIG)’, IEEE.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D. & Riedmiller, M. (2013), ‘Playing atari with deep reinforcement learning’, *arXiv preprint arXiv:1312.5602* .
- Nichol, A., Achiam, J. & Schulman, J. (2018), ‘On first return, then explore: Exploiting the learned structure of the environment’, *OpenAI Blog* .
URL: <https://openai.com/blog/retro-contest>

PyLessons (2020), ‘Introduction to advantage actor-critic method (a2c)’, *PyLessons*

URL: <https://pylessons.com/A2C-reinforcement-learning>

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017), ‘Proximal policy optimization algorithms’, *arXiv preprint arXiv:1707.06347*.

URL: <https://arxiv.org/pdf/1707.06347>

Sutton, R. S. & Barto, A. G. (2018), *Reinforcement Learning: An Introduction*, 2nd edn, MIT press.

URL: <http://incompleteideas.net/book/RLbook2020.pdf>

Togelius, J., Yannakakis, G. N., Stanley, K. O. & Browne, C. (2011), ‘Search-based procedural content generation: A taxonomy and survey’, *IEEE Transactions on Computational Intelligence and AI in Games*.

Vinyals, O., Babuschkin, I., Czarnecki, W. M. et al. (2019), ‘Grandmaster level in starcraft ii using multi-agent reinforcement learning’, *Nature*.

Yerremshetty, R. (2025), ‘Understanding exploration vs. exploitation in reinforcement learning (rl)’, *Nomidl*.

URL: <https://www.nomidl.com/generative-ai/exploration-vs-exploitation-rl/>

Declaration on the use of generative Artificial Intelligence (AI) systems

Name | Family Name **Megrelishvili**

Vorname | First Name **Sandro**

39851214

Bachelor Thesis

Matrikelnummer | Student ID Number

Titel Prüfungsarbeit | Title of the exam

I have used the following artificial intelligence (AI)-based tools in the creation of my work:

1. ChatGPT
2. Grammarly
3. ...

I further declare that

- ☒ I have actively informed myself about the performance and limitations of the above-mentioned AI tools,
- ☒ I have marked the passages taken from the above-mentioned AI tools,
- ☒ I have checked that the content generated with the help of the above-mentioned AI tools and adopted by me is actually accurate,
- ☒ I am aware that, as the author of this work, I am responsible for the information and statements made in it.

I have used the AI-based tools mentioned above as shown in the table below.

AI-based support tool	Usage	Parts of the work affected	Remarks
ChatGPT (free Version 4o), OpenAI	Helped with LaTeX formatting.	All chapters	
Grammarly (free Version), Grammarly	spelling and grammar correction	All chapters	
...	...	...	
...	...	...	

Place, date, signature of the student

Berlin, Germany

10/07/2025

